Progetto Milano - Cortina 2026

Prova Progettuale di Analisi e Progettazione del Software A.A. 2025 / 2026



Garancini Andrea

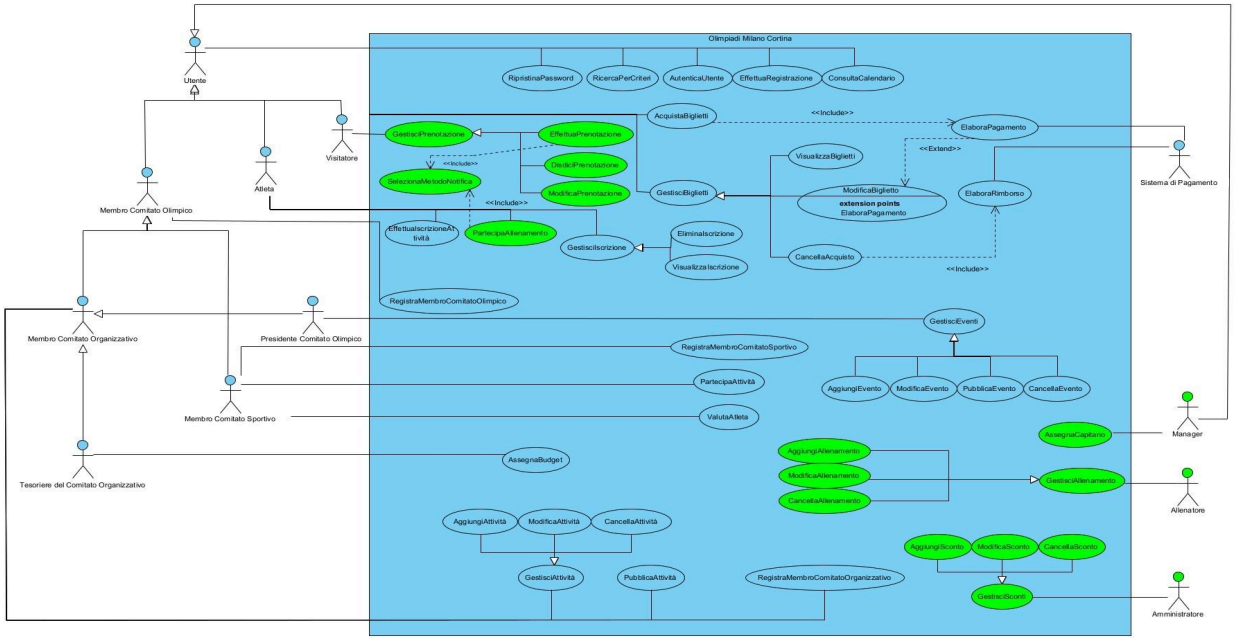
06/06/2026

1. Diagramma dei Casi d'Uso **4**
2. Casi D'Uso In Formato Dettagliato **5**
CASO D'USO DETTAGLIATO: "AutenticaUtente" 5
Caso d'uso esteso: AggiungiAllenamento 8
Caso D'Uso Esteso: CancellaAllenamento 10
Caso D'Uso Esteso: EffettuaPrenotazione 12
Caso D'Uso Esteso: PartecipaAllenamento 14
3. Diagrammi di Sequenza di Sistema **16**
AutenticaUtente 16
AggiungiAllenamento 17
CancellaAllenamento 18
EffettuaPrenotazione 19
PartecipaAllenamento 20
4. Operazioni di Sistema - Contratti **21**
Contratto InserisciAttività 21
Contratto PubblicaAttività 21
Contratto: AggiungiAllenamento 22
Contratto: cancellaAllenamento 23
Contratto EffettuaPrenotazione 24
5. Modello di Dominio **25**
Diagramma di Attività: GestioneAllenamenti 26
Diagramma delle Attività: Prenotazione Biglietto 28
Diagramma delle Attività: AutenticaUtente 29
7. Diagramma dei Package **30**
8. Diagramma delle Classi di Progettazione **31**
9. Diagrammi di Sequenza **35**
AutenticaUtente 35
AggiungiAllenamento 35
CancellaAllenamento 36
EffettuaPrenotazione 37
PartecipaAllenamento 38
10. Macchine a Stati Finiti **39**
Macchina a stati finiti dell'oggetto Biglietto 39
Macchina a Stati Finiti dell'Oggetto Attività 40
11. Diagrammi di Comunicazione **41**
Diagramma di Comunicazione: InserisciAttività 41
Diagramma di Comunicazione: PubblicaAttività 42
12.Pattern GRASP e GOF **43**
Design Pattern 43
Singleton 43
Pattern GRASP 43
Information Expert 43
Creator 44
Low-Coupling 44

High Cohesion 45
Controller 45
Polymorphism 46

1. Diagramma dei Casi d’Uso

Autore: Cestrone, Gaspani



2. Casi D’Uso In Formato Dettagliato

Parte comune:

CASO D’USO DETTAGLIATO: “AutenticaUtente”

Nome Caso D’Uso: AutenticaUtente
Portata: Progetto Olimpiadi Milano Cortina 2026
Livello: Obiettivo Utente
Attore primario: Utente

Parti interessate e interessi:

- Utente: vuole autenticarsi in maniera semplice ed intuitiva per diventare visitatore e per poter effettuare operazioni che richiedono l’autenticazione (ad esempio l’acquisto e la gestione dei biglietti).
- Membro del Comitato Organizzativo: vuole autenticarsi al Sistema per effettuare le operazioni riservate relative al suo ruolo (ad esempio gestione delle attività, ecc.)
- Membro del Comitato Sportivo: vuole autenticarsi al Sistema per effettuare le operazioni riservate relative al suo ruolo (ad esempio valutazione degli atleti, ecc.)
- Atleta: vuole autenticarsi al Sistema per effettuare le operazioni riservate relative al suo ruolo (ad esempio iscrizione alle attività, ecc.)

Precondizioni:

- L’Utente non è autenticato.

Garanzia di successo:

- L’Utente viene autenticato correttamente.
- Il Sistema consente l’accesso alle funzionalità riservate in base al ruolo dell’Utente.

Scenario principale di successo:

1. L’Utente entra nella pagina di login
2. L’Utente inserisce il nome utente nella sezione apposita
3. L’Utente inserisce la password nella sezione apposita
4. L’Utente conferma i dati inseriti
5. Il Sistema verifica i dati inseriti dall’Utente
6. Il Sistema autentica l’Utente

Estensione:

5a. Le credenziali inserite dall'Utente non sono corrette
1. Il Sistema segnala un errore
2. Il Sistema chiede all'Utente di riprovare

2a. L'Utente accetta

2a.1. Si ripetono i passi dal punto 2

2b. L'Utente rifiuta

2b.1. Il Sistema riporta l'Utente alla pagina iniziale

5b. Numero massimo di tentativi superato
1. Il Sistema blocca temporaneamente l'account
2. Il Sistema informa l'Utente del blocco temporaneo
3. Il Sistema registra l'evento per motivi di sicurezza

5c. Le credenziali inserite non sono associate a nessun account
1. Il Sistema segnala un errore
2. Il Sistema propone all'Utente di registrarsi

2a. L'Utente accetta

2a.1. Viene avviato il caso d'uso "EffettuaRegistrazione"

2b. L'Utente rifiuta

2b.1. Il Sistema riporta l'Utente alla pagina iniziale

5d. L'Utente dimentica la password
1. L'Utente seleziona "Password dimenticata"
2. Il Sistema avvia il caso d'uso "RecuperaPassword"

Requisiti speciali:

- Interfaccia utente conforme alle linee guida WCAG per l'accessibilità
- L'interfaccia di accesso deve essere adattata in base al dispositivo dell'utente
- Possibilità di scelta della lingua in modo che ogni utente non abbia problemi nella comprensione durante l'accesso
- Limite di 5 errori da parte dell'utente prima di un blocco temporaneo per motivi di sicurezza, oltre i quali si verificherà un blocco temporaneo di 15 minuti
- La comunicazione durante un tentativo di accesso deve essere cifrata per motivi di sicurezza
- Il sistema deve supportare autenticazioni concorrenti multiple senza degrado significativo delle prestazioni.

Elenco delle variabili tecnologiche e dei dati:

- Credenziali di accesso
- Numero di tentativi falliti consecutivi

Frequenza di ripetizione: Alta frequenza, ripetuto ogni volta che un utente non autenticato vuole accedere a funzionalità che richiedono l'autenticazione.
Varie: nessuna

Autore: Cestrone

Caso d’uso esteso: AggiungiAllenamento

Nome del Caso d’Uso: AggiungiAllenamento.
Portata: Progetto Olimpiadi Milano Cortina 2026.
Livello: Obiettivo Utente.
Attore Primario: Allenatore.
Parti interessate e interessi:

- Atleti (registrati):
    - L'atleta registrato vuole poter consultare informazioni complete, corrette e aggiornate sulle sessioni di allenamento disponibili, in modo da poter valutare una futura partecipazione.

Pre-Condizioni:

- L’allenatore è correttamente autenticato nel sistema.

Garanzia di Successo: L’allenatore inserisce correttamente le informazioni sull’allenamento, che viene pubblicato e reso disponibile agli atleti.
Scenario Principale di Successo:

1. L’utente Allenatore richiede la creazione di una nuova sessione di allenamento.
2. Il Sistema mostra il modulo per la creazione della sessione di allenamento.
3. L’utente Allenatore inserisce i dati relativi all’allenamento (data, ora, luogo, maxAtleti, tema).
4. L’utente Allenatore conferma la creazione della sessione di allenamento.
5. Il Sistema verifica la validità dei dati inseriti.
6. Il Sistema registra e rende pubblico l’allenamento.
7. Il Sistema notifica all’utente Allenatore l’avvenuta creazione della sessione di allenamento.

Estensioni:
5a. Uno o più dati mancanti o non validi
5a.1. Il Sistema segnala all’utente Allenatore i dati mancanti o non validi.
5a.2. Il Sistema richiede la correzione dei dati.
5a.3. L’utente Allenatore modifica i dati segnalati.
5a.4. Il caso d’uso riprende dal punto 5 dello Scenario Principale di Successo.
Requisiti Speciali:

- Il Sistema deve impedire la creazione di una sessione di allenamento in una data e ora già trascorse.
- Il numero massimo di Atleti deve essere un valore intero maggiore di zero.
- La sessione di allenamento deve contenere almeno un tema.
- il Sistema deve notificare immediatamente l'utente Allenatore dell'avvenuta creazione della sessione di allenamento.

Elenco delle variabili tecnologiche e dei Dati:

- Data e ora dell'allenamento.
- luogo di svolgimento.
- numero massimo di Atleti.
- Uno o più temi dell'allenamento.
- Formato di visualizzazione di data e ora.

Frequenza di Ripetizione: occasionale, in base alla pianificazione delle sessioni di allenamento.

Varie: /

Autore: Colombi

Caso D'Uso Esteso: CancellaAllenamento

Nome del Caso D'Uso: CancellaAllenamento.
Portata: Progetto Olimpiadi Milano Cortina 2026.
Livello: Obiettivo Utente.
Attore Primario: Allenatore.
Parti Interessate e Interessi:

- Atleta (quelli che partecipano all'allenamento):
    - L'Atleta che partecipa all'allenamento vuole essere notificato in caso di cancellazione tramite il recapito che preferisce (SMS o Mail). Inoltre, vuole la garanzia che l'allenamento a cui partecipa non venga cancellato nell'ora precedente al suo inizio.

Pre-condizioni:

- L'Allenamento selezionato deve avere come data e orario più un'ora di distanza dalla data e orario in cui viene richiesta la cancellazione da parte dell'Allenatore.
- L'Atleta deve aver selezionato un recapito.
- L'Allenamento che si sta provando a cancellare deve esistere.

Garanzia di Successo: L'Allenamento viene cancellato e gli atleti iscritti sono notificati, tramite il recapito da loro selezionato, della cancellazione.
Scenario Principale di Successo:

1. L'Utente Allenatore seleziona un allenamento
2. L'Utente Allenatore tenta di cancellare l'allenamento
3. Il Sistema verifica che il vincolo temporale sia rispettato
4. Il Sistema cancella l'allenamento
5. Il Sistema notifica gli Utenti Atleta della cancellazione tramite il recapito da loro selezionato
6. Il Sistema informa l'Utente Allenatore della cancellazione dell'allenamento

Estensioni:
3a. Il vincolo temporale non è rispettato
3a.1. Il Sistema avvisa l'Utente Allenatore che manca meno di un'ora all'inizio dell'allenamento, rifiutando la cancellazione
5a. Il Sistema non riesce a notificare uno o più Utenti Atleta
5a.1 Il Sistema avvisa l'Utente Allenatore fornendogli una lista di Utenti Atleta che non è riuscito a notificare
Requisiti Speciali:

- Il Sistema deve mantenere coerenza in tempo reale degli allenamenti cancellati per tutti gli utenti che hanno il permesso di vederli.

- Il Sistema deve notificare il più velocemente possibile gli Atleti che partecipano ad un allenamento quando viene cancellato.

Elenco delle Variabili Tecnologiche e dei Dati:

- Preferenza del recapito di tutti gli Atleti che partecipano all'Allenamento cancellato
- Lista degli Atleti che partecipano all'Allenamento cancellato
- Data e Ora attuale
- Servizio di invio notifiche

Frequenza di Ripetizione: bassa/occasionale

Varie: /

Autore: Garancini

Caso D'Uso Esteso: EffettuaPrenotazione

Nome del Caso D'Uso: EffettuaPrenotazione
Portata: Progetto Olimpiadi Milano Cortina 2026.
Livello: Obiettivo Utente.
Attore Primario: Visitatore
Parti Interessate e Interessi:

- Visitatore: Il Visitatore effettua la prenotazione per ricevere una notifica ogni qual volta si libera un posto o vengono pubblicati i biglietti di una specifica attività o evento
- Sistema: Il Sistema deve inviare una notifica ai visitatori che hanno effettuato una prenotazione

Pre-condizioni:

- Il visitatore deve avere effettuato correttamente il login
- I biglietti per l'attività o per l'evento di interesse non sono ancora stati pubblicati oppure sono esauriti

Garanzia di Successo: La Prenotazione del Visitatore viene correttamente salvata nel sistema e il Visitatore riceve conferma dell'avvenuta registrazione della Prenotazione

Scenario Principale di Successo:

1. Il Visitatore clicca sul pulsante per richiedere una prenotazione
2. Il Sistema apre la schermata relativa all'inserimento di una nuova prenotazione
3. Il Visitatore seleziona l'attività o l'evento di interesse
4. Il Visitatore inserisce il recapito a cui vuole che sia inviata la notifica
5. Il Visitatore clicca il pulsante di conferma della prenotazione
6. Il Sistema verifica i dati inseriti
7. Il Sistema salva la prenotazione
8. Il Sistema conferma inserimento prenotazione

Estensioni:

6a. L'Utente inserisce un recapito non valido.
6a.1. Il Sistema chiede di reinserire il recapito corretto.
6a.1.1. Il Visitatore inserisce correttamente il recapito.
6a.1.2. Il Sistema procede dal punto 5.
6a.2. Il Visitatore rinuncia all'inserimento del recapito.
6a.2.1. Il Sistema annulla la prenotazione e torna alla schermata iniziale.

Requisiti Speciali:

- Il Sistema deve accettare un numero indeterminato di prenotazioni per lo stesso evento o la stessa attività
- Il Sistema deve provvedere al salvataggio dei dati in modalità sicura, essendo i dati di contatto dati sensibili

Elenco delle Variabili Tecnologiche e dei Dati:

- Recapito di contatto del visitatore (Telefono per SMS o indirizzo mail)

Frequenza di Ripetizione: Molto elevata, quasi ininterrotta
Varie: Nessuna

Autore: Gaspani

Caso D'Uso Esteso: PartecipaAllenamento

Nome del Caso D'Uso: PartecipaAllenamento.
Portata: Progetto Olimpiadi Milano Cortina 2026.
Livello: Obiettivo Utente.
Attore Primario: Atleta.
Parti Interessate e Interessi:

- Atleta: Vuole potersi iscrivere a una sessione di allenamento pianificata dal proprio allenatore, purché ci sia disponibilità di posti e l'orario non si sovrapponga con altri suoi impegni o iscrizioni ad attività o eventi olimpici.
- Allenatore: Vuole che il numero di atleti iscritti non superi il limite massimo prefissato per la sessione e desidera che la lista dei partecipanti sia aggiornata accuratamente per l'organizzazione logistica della sessione

Pre-condizioni:

- L'Atleta deve essere autenticato nel sistema.
- La sessione di allenamento selezionata deve essere già stata creata e pubblicata dall'allenatore.
- La sessione di allenamento non deve essere ancora iniziata.

Garanzia di Successo: l'Atleta viene registrato con successo all'interno della sessione di allenamento selezionata. Il numero di posti disponibili per la sessione viene decrementato e il sistema conferma l'avvenuta iscrizione.
Scenario Principale di Successo:

1. L'Utente Atleta visualizza la lista delle sessioni di allenamento disponibili organizzate dal proprio allenatore.
2. L'Utente Atleta seleziona la sessione di allenamento a cui desidera partecipare.
3. Il Sistema verifica che la sessione non abbia raggiunto il numero massimo di atleti consentiti.
4. Il Sistema verifica che l'Utente Atleta non sia gia iscritto alla medesima sessione o che non abbia altre sovrapposizioni orarie.
5. Il Sistema registra l'iscrizione dell'Atleta alla sessione di allenamento.
6. Il Sistema aggiorna il conteggio dei partecipanti per quella sessione.
7. Il Sistema informa l'Utente Atleta della corretta registrazione all'allenamento.

Estensioni:

3a. Il numero massimo di atleti per la sessione è già stato raggiunto

3a.1. Il Sistema avvisa l'Utente Atleta che la sessione è al completo, rifiutando
l'iscrizione.

4a. L'Utente Atleta risulta già iscritto alla sessione di allenamneto o ha un conflitto di orario

4a.1 Il Sistema segnala l'errore di ridondanza o di sovrapposizione oraria all'Utente Atleta e nega l'operazione.

Requisiti Speciali:

- Il Sistema deve mantenere la disponibilità dei posti della sessione di allenamento in tempo reale, garantendo la consistenza dei dati in caso di accessi concorrenziali multipli(Evitando l'overbooking oltre il limite massimo fissato dall'allenatore).
- L'interfaccia deve permettere all'atleta di distinguere chiaramente la tipologia della sessione prima di effettuare l'iscrizione

Elenco delle Variabili Tecnologiche e dei Dati:

- Identificativo univoco dell'Utente Atleta
- Identificativo univoco della Sessione di Allenamento selezionata.
- Dati della sessione: Data, Ora di inizio, Luogo, Numero massimo di partecipanti, Tipologia
- Contatore corrente degli atleti registrati per la specifica sessione
- Data e Ora attuale del server per la convalida dei vincoli temporali

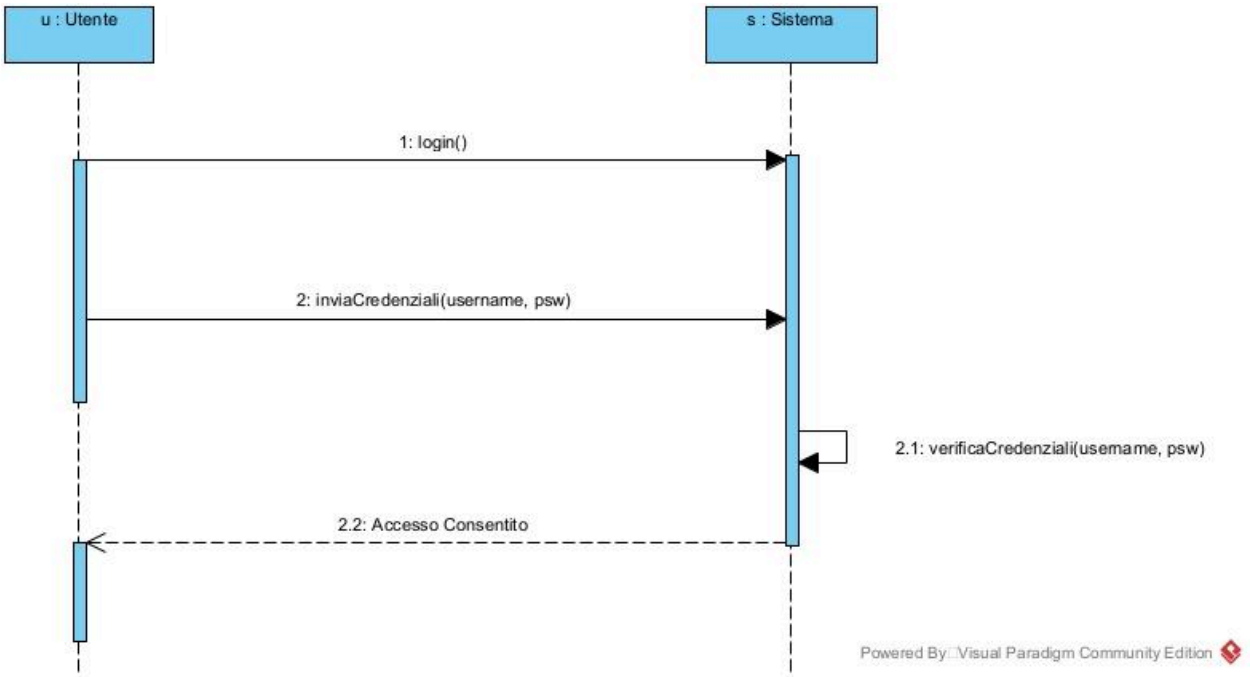
Frequenza di Ripetizione: Media/Alta (soprattutto nei periodi di ridosso delle gare)

Varie: /

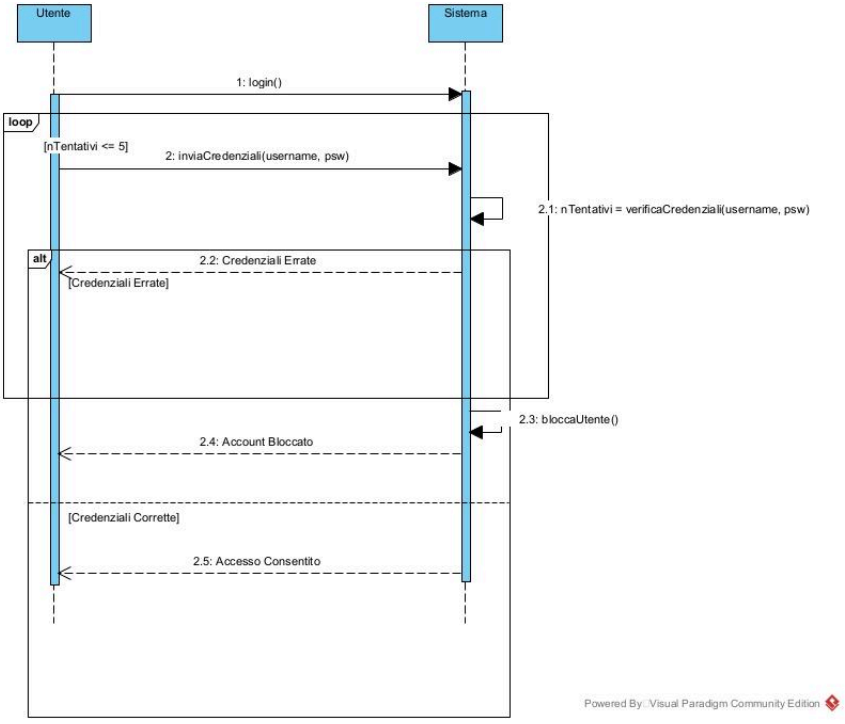
3. Diagrammi di Sequenza di Sistema

AutenticaUtente

Autore: in comune

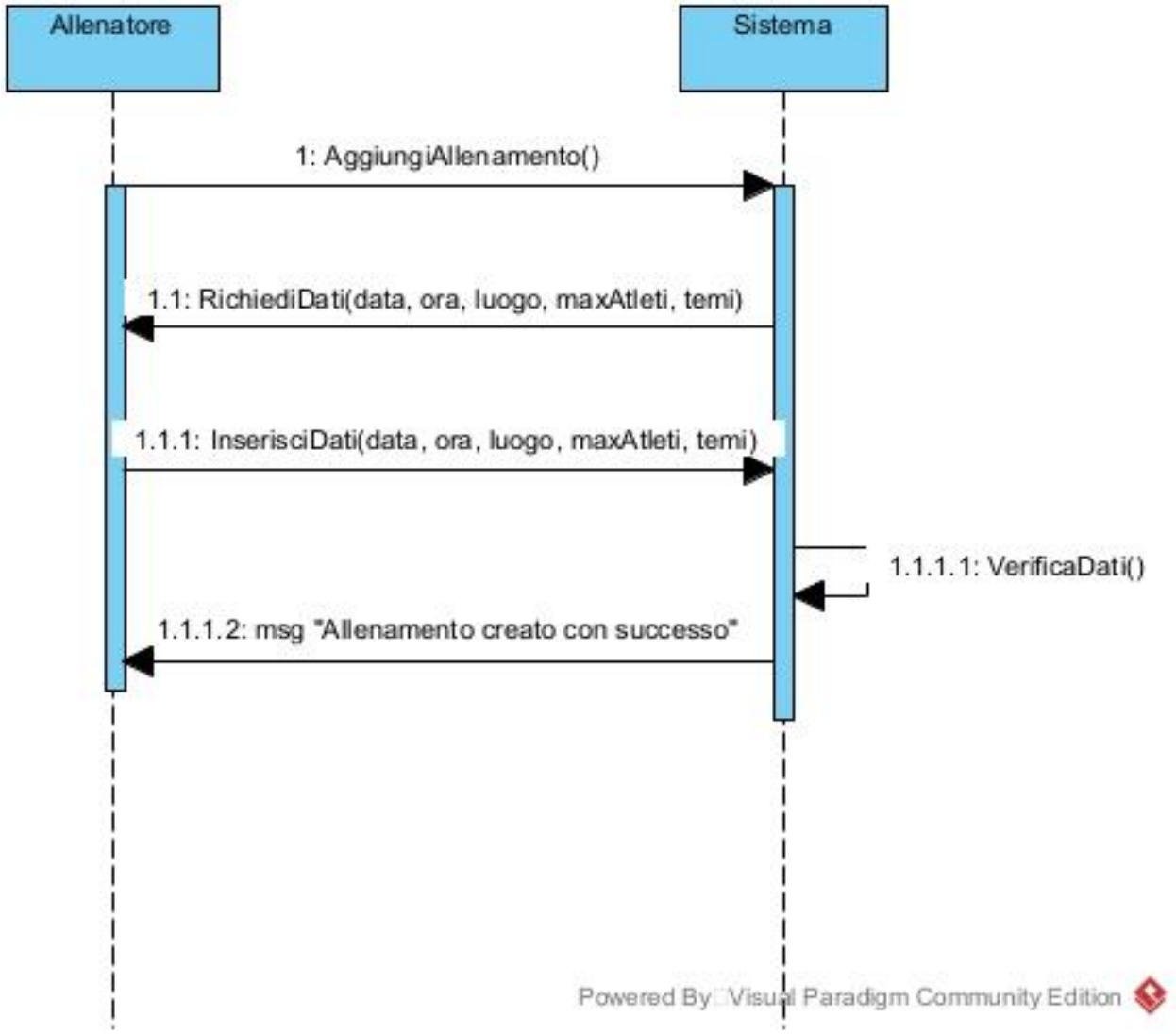


Data la semplicità del diagramma di sequenza relativo allo scenario principale, abbiamo deciso di rappresentare anche il diagramma di sequenza di un'estensione notevole in un diagramma separato.



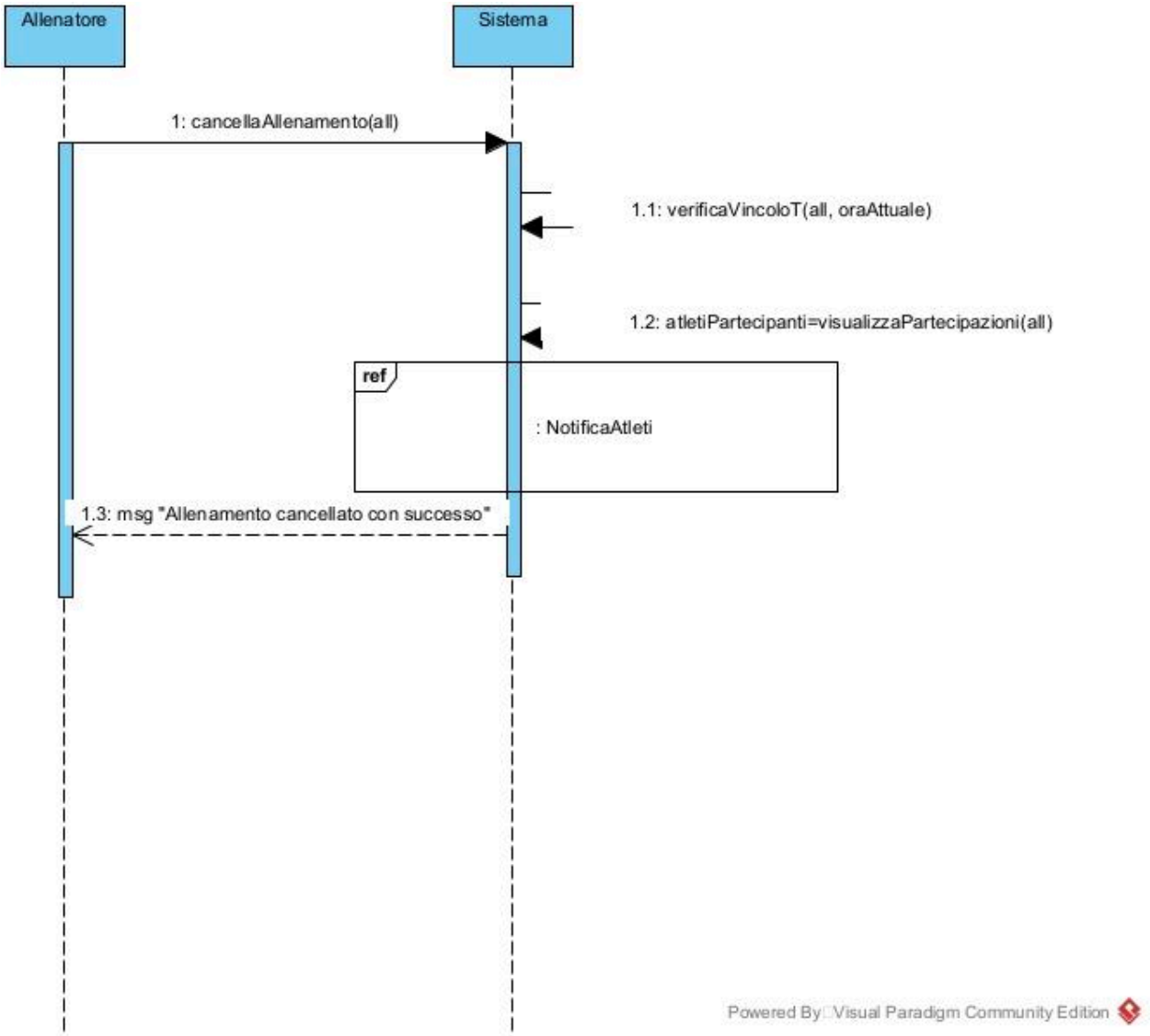
AggiungiAllenamento

Autore: Cestrone



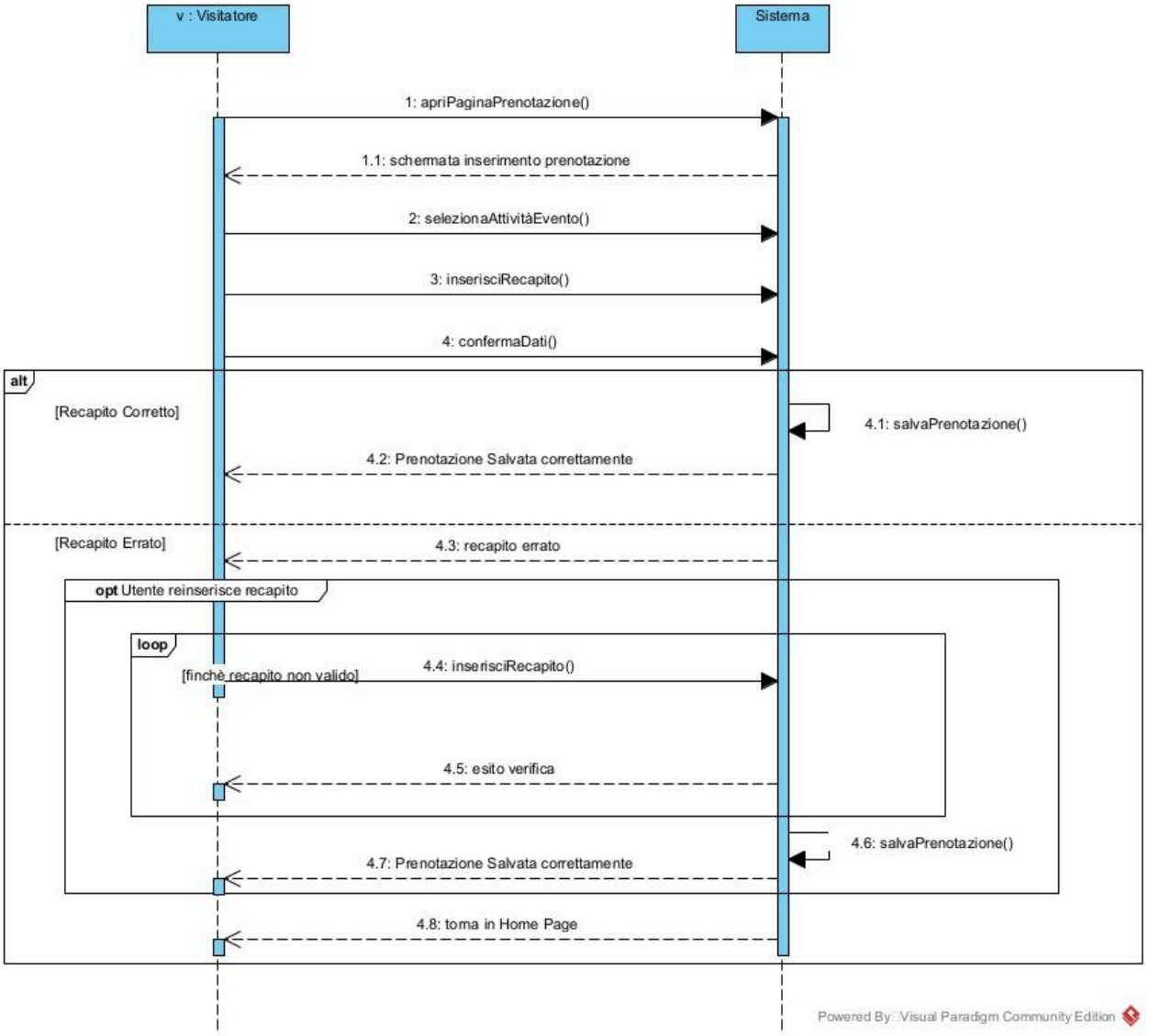
CancellaAllenamento

Autore: Colombi



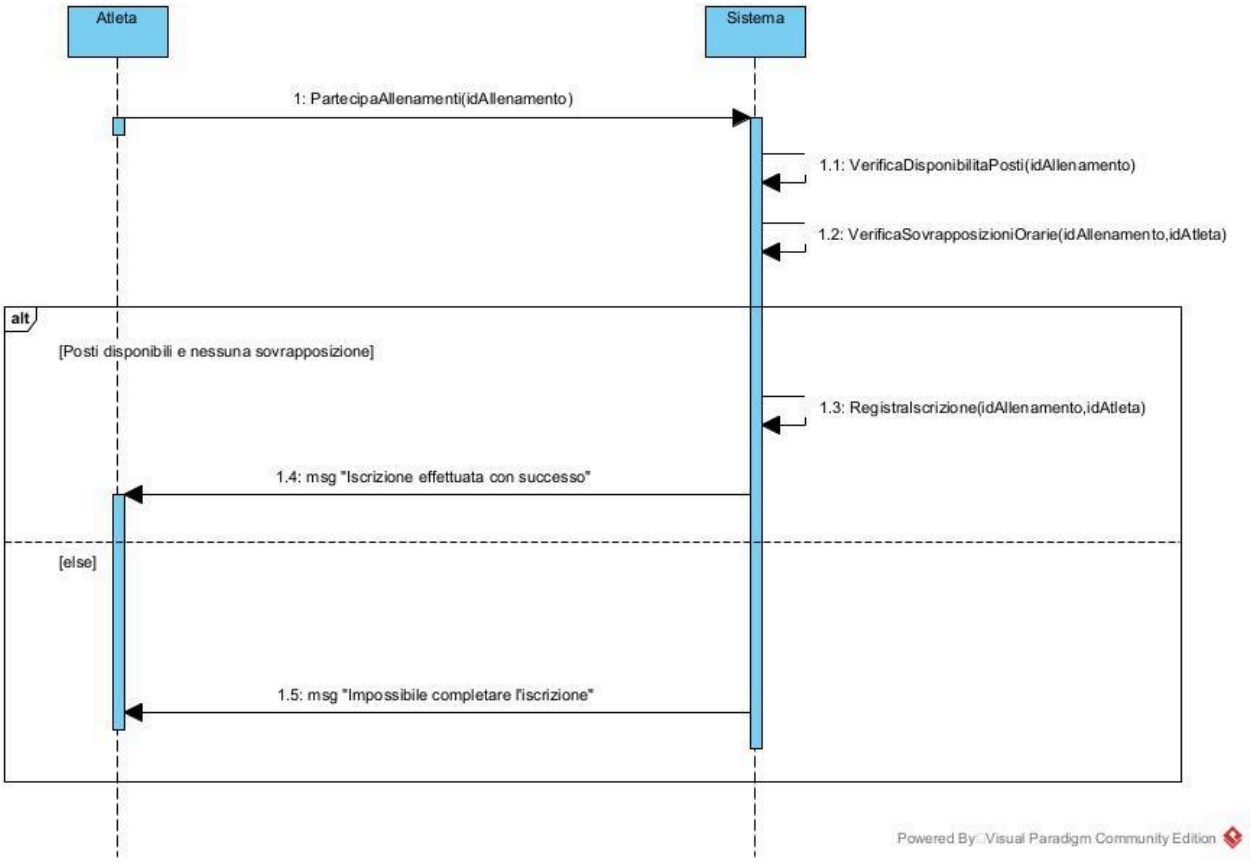
EffettuaPrenotazione

Autore: Garancini



PartecipaAllenamento

Autore: Gaspani



4. Operazioni di Sistema - Contratti

Parte comune:

Contratto InserisciAttività

Operazione: InserisciAttività(nome: string, data: date, luogo: string, oraInizio: time)
Riferimenti caso d'uso: GestisciAttività
Precondizioni: Il membro del Comitato Organizzativo ha effettuato correttamente il login
Postcondizioni:

- è stata creata un'istanza 'att' di Attività (creazione di oggetto)
- att.nome è diventato nome (modifica di attributo)
- att.data è diventato data (modifica di attributo)
- att.luogo è diventato luogo (modifica di attributo)
- att.oraInizio è diventato oraInizio (modifica di attributo)
- att.stato è diventato "Non Pubblicata" (modifica attributi)

Contratto PubblicaAttività

Operazione: PubblicaAttività(att: Attività)
Riferimenti caso d'uso: PubblicaAttività
Precondizioni:

- Il membro del Comitato Organizzativo ha effettuato correttamente il login
- L'attività che si vuole pubblicare è già stata creata
- L'attributo 'stato' dell'attività' è impostato su "non pubblicata"

Postcondizioni:

- att.stato è diventato "Pubblicata" (modifica attributo)
- è stata creata una associazione tra l'istanza di Attività 'att' e l'istanza di Calendario 'c' (formazione di collegamento)

Autore: Cestrone

Contratto: AggiungiAllenamento

Operazione: AggiungiAllenamento(data: Date, ora: Time, luogo: String, maxAtleti: int; tema: List<string>).
Riferimenti Caso d'Uso: AggiungiAllenamento.
Pre-condizioni:

- L'utente Allenatore è autenticato nel sistema.
- Nel Sistema non è già presente la sessione di Allenamento che si desidera aggiungere.

Post-condizioni:

- E' stata creata una nuova istanza all:Allenamento ***(Creazione di Oggetto)*****.**
- Alla nuova istanza 'all' sono assegnati attributi all.data, all.ora, all.luogo, all.maxAtleti, all.tema, con valore rispettivamente data, ora, luogo, maxAtleti, tema. ***(Modifica di Attributi)***.
- La nuova istanza 'all' è associata all'utente Allenatore che l'ha creata ***(Creazione di Associazione)***.
- La nuova istanza 'all' è memorizzata nel Sistema ***(Persistenza dell'Oggetto)***.

Autore: Colombi

Contratto: cancellaAllenamento

Operazione: cancellaAllenamento(all:Allenamento)
Riferimenti: Caso d'Uso: CancellaAllenamento
Pre-condizioni:

- Un Utente di tipo Allenatore ha richiesto la cancellazione di un Allenamento
- Il vincolo temporale (DataAttuale< all.data oppure OraAttuale-all.ora> 1) è rispettato

Post-condizioni:

- E' stata rimossa l'istanza all : Allenamento (cancellazione di oggetto)
- Sono state rimosse tutte le istanza di PartecipazioneAll riguardanti all (cancellazione di oggetto)
- Sono stati rimossi tutti i collegamenti tra le istanze di Allenamento e PartecipazioneAll riferiti a all (rottura di collegamento)
- Sono stati rimossi tutti i collegamenti tra le istanze di Atleta e PartecipazioneAll riferiti a all (rottura di collegamento)

Autore: Garancini

Contratto EffettuaPrenotazione

Operazione: effettuaPrenotazione(recapito: string, b: Biglietto) : boolean
Riferimenti caso d'uso: EffettuaPrenotazione
Precondizioni:

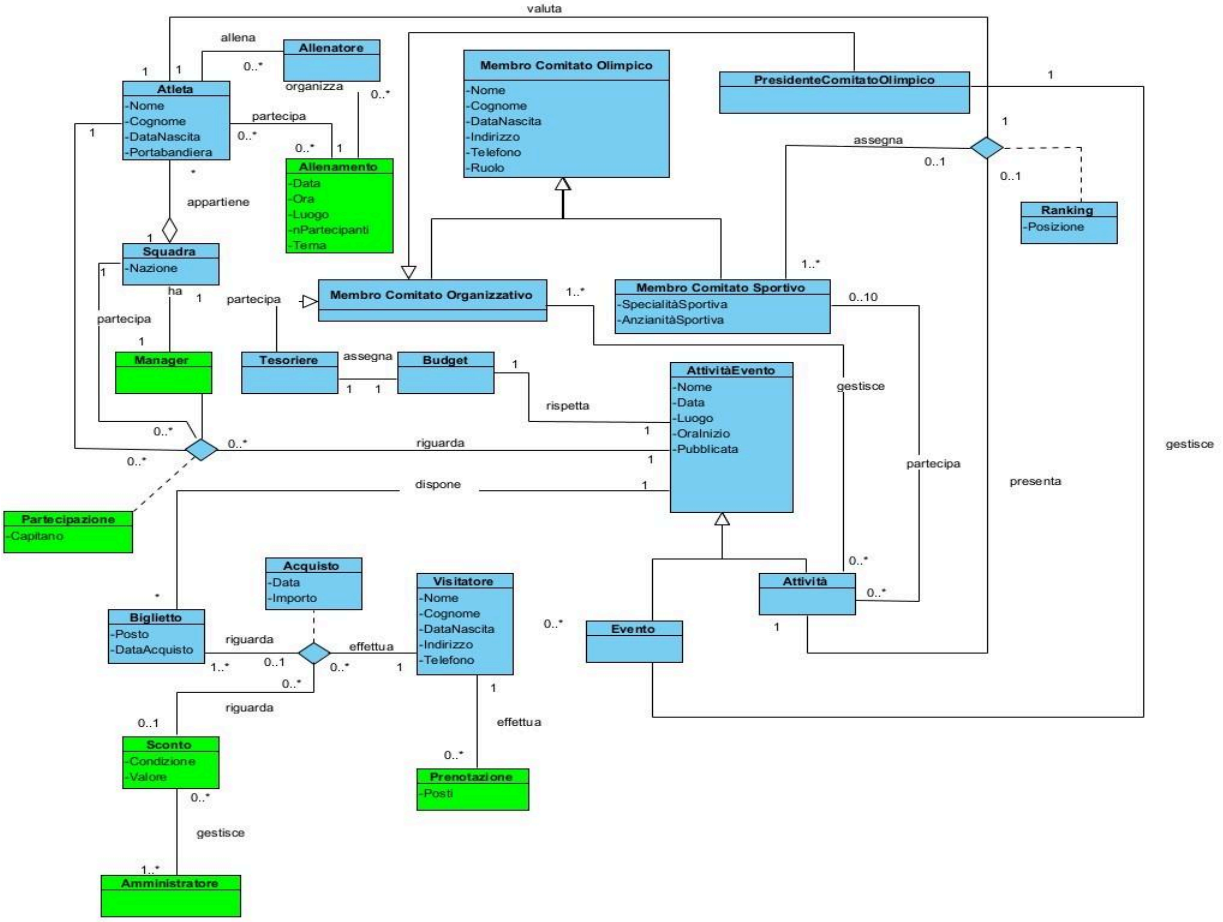
- Il Visitatore ha effettuato correttamente il login
- L'Attività o l'Evento per cui si effettua la prenotazione non ha più biglietti disponibili oppure i biglietti non sono ancora stati pubblicati

Postcondizioni:

- è stata creata un'istanza "p" di Prenotazione (creazione di oggetto)
- p.recapito è diventato recapito (modifica di attributo)

5. Modello di Dominio

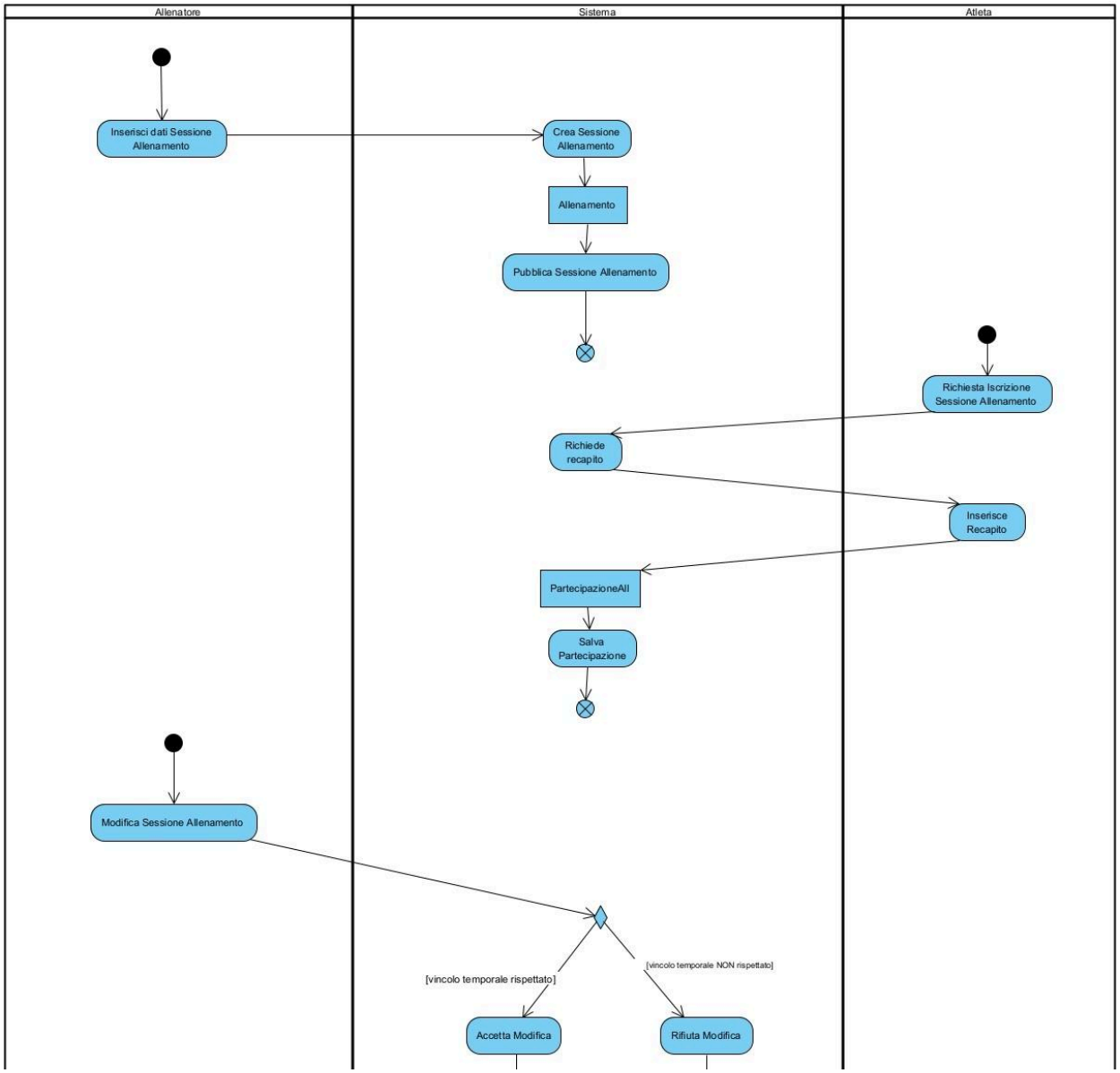
Autori: Colombi, Garancini



6. Diagrammi di Attività

Parte comune:

Diagramma di Attività: GestioneAllenamenti



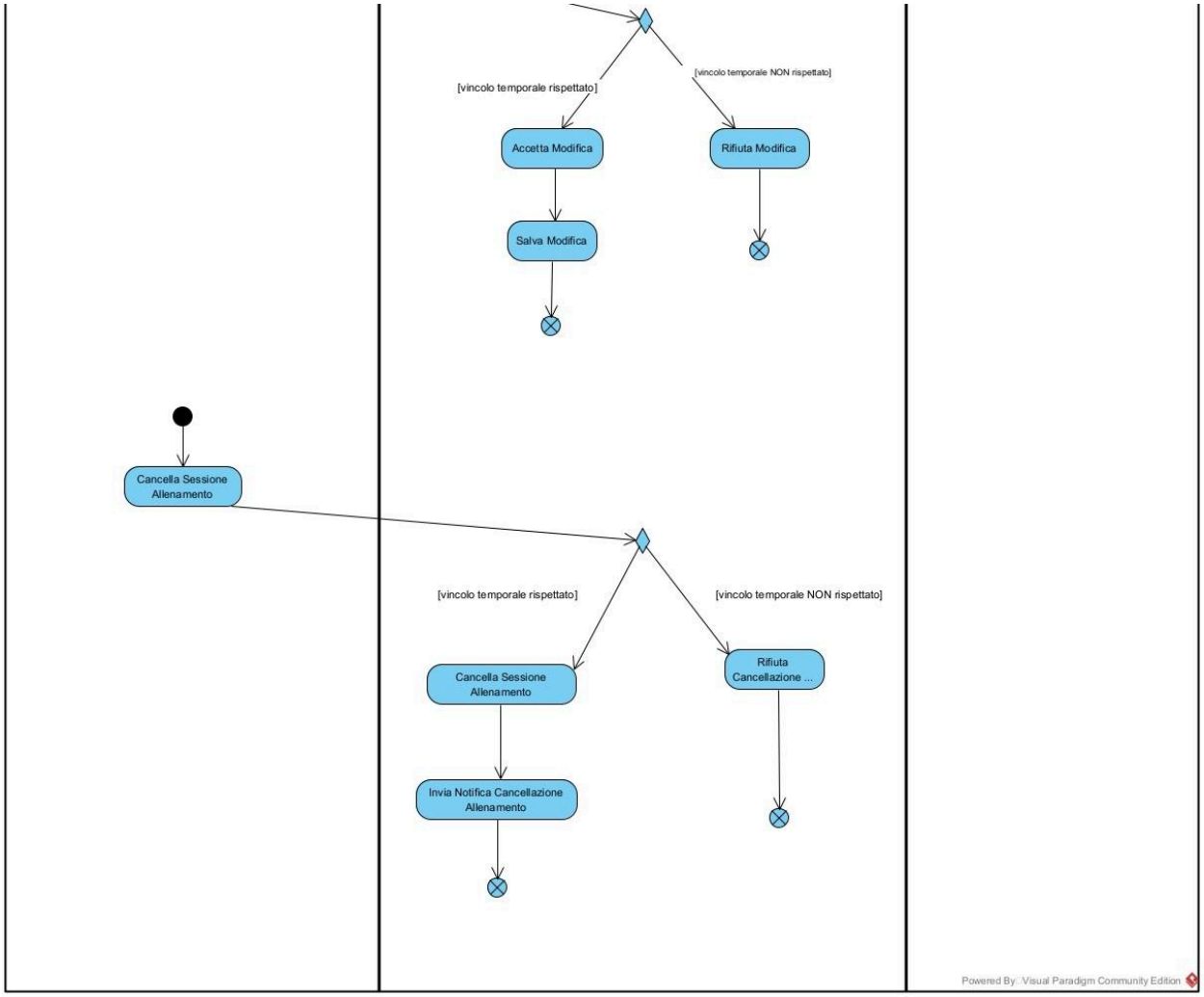

[vincolo temporale rispettato]
[vincolo temporale NON rispettato]
Accetta Modifica
Rifiuta Modifica
Salva Modifica
Cancella Sessione Allenamento
[vincolo temporale rispettato]
[vincolo temporale NON rispettato]
Cancella Sessione Allenamento
Rifiuta Cancellazione ...
Invia Notifica Cancellazione Allenamento
Powered By: Visual Paradigm Community Edition

Diagramma delle Attività: Prenotazione Biglietto

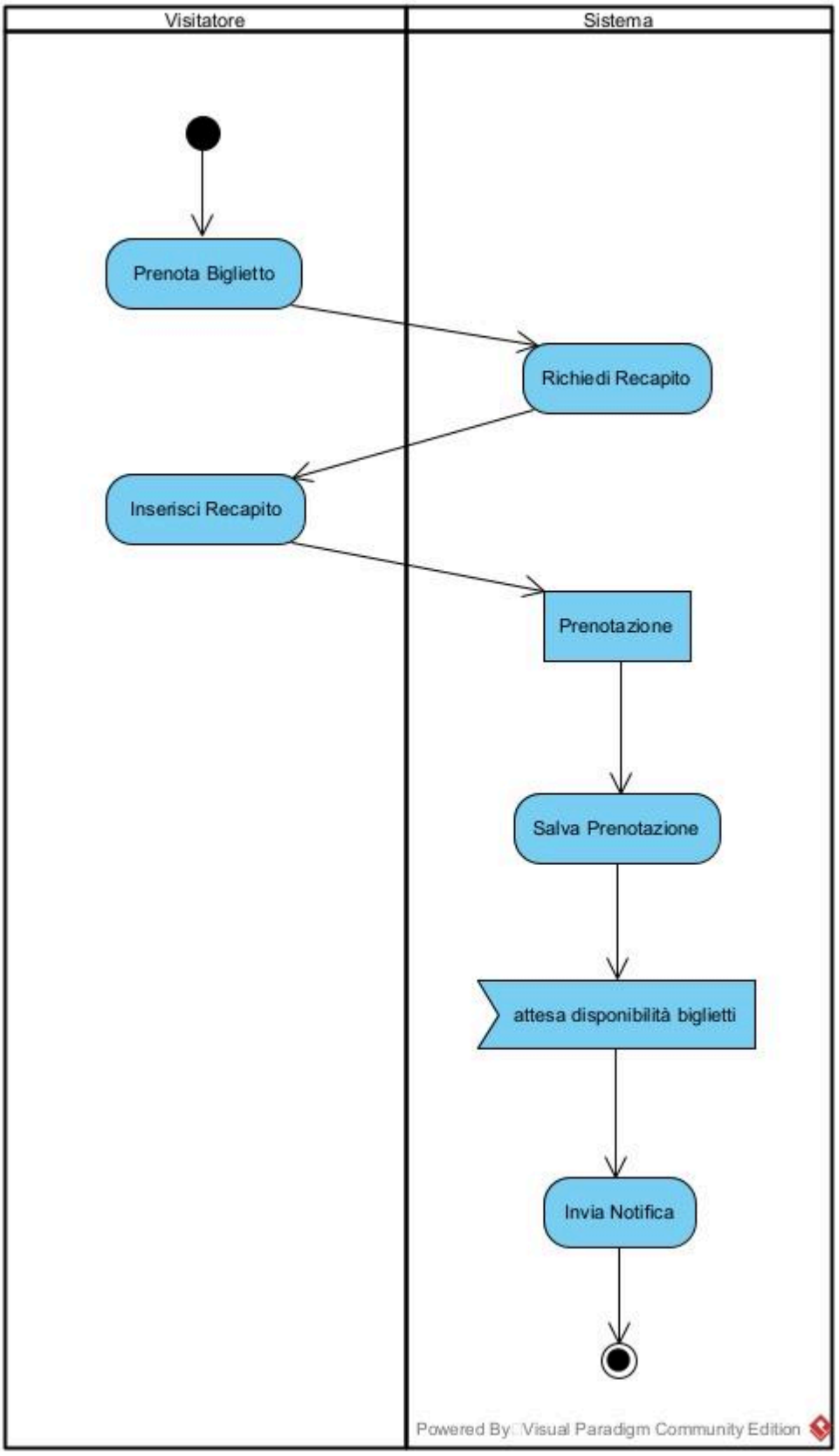
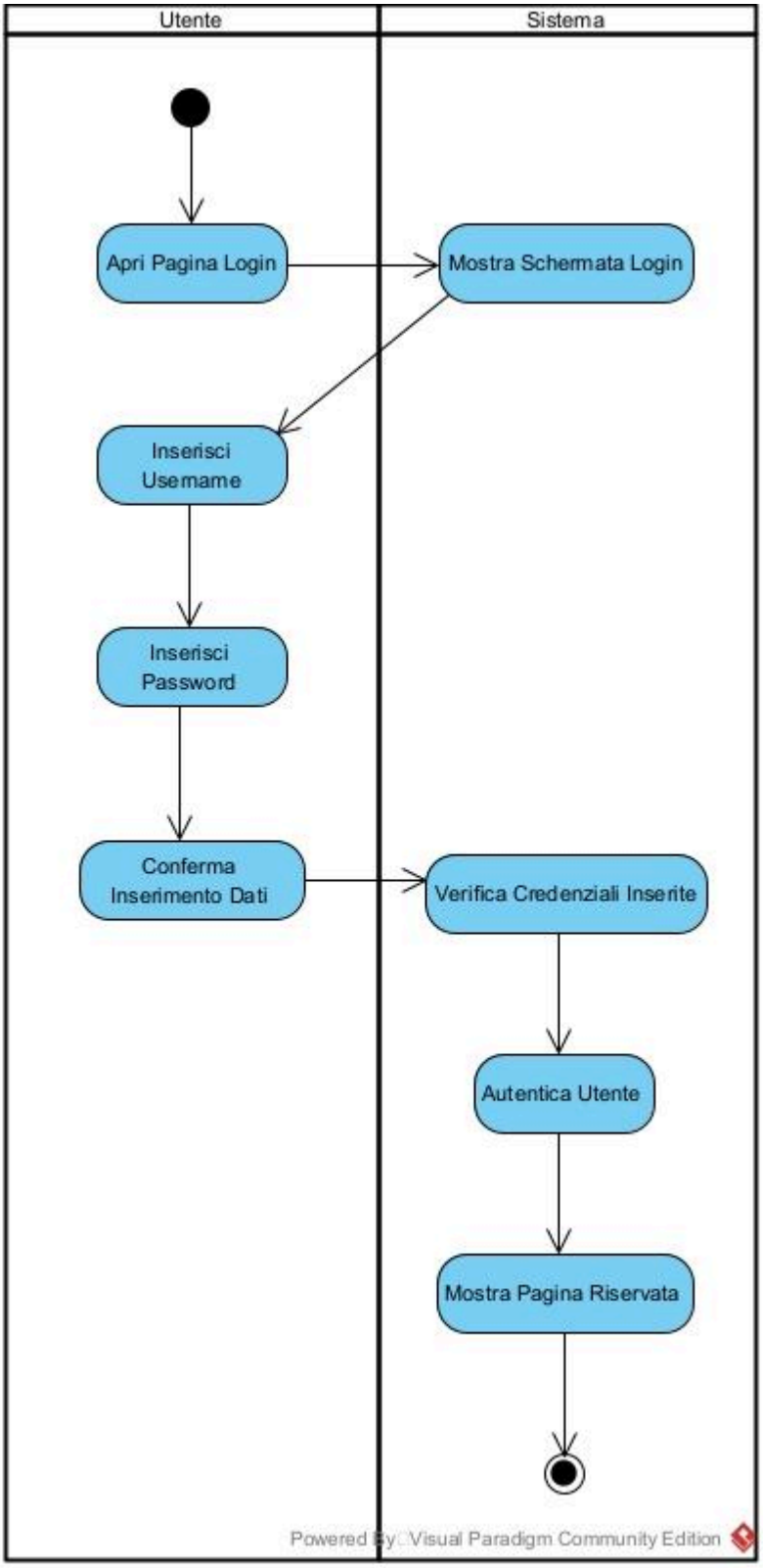
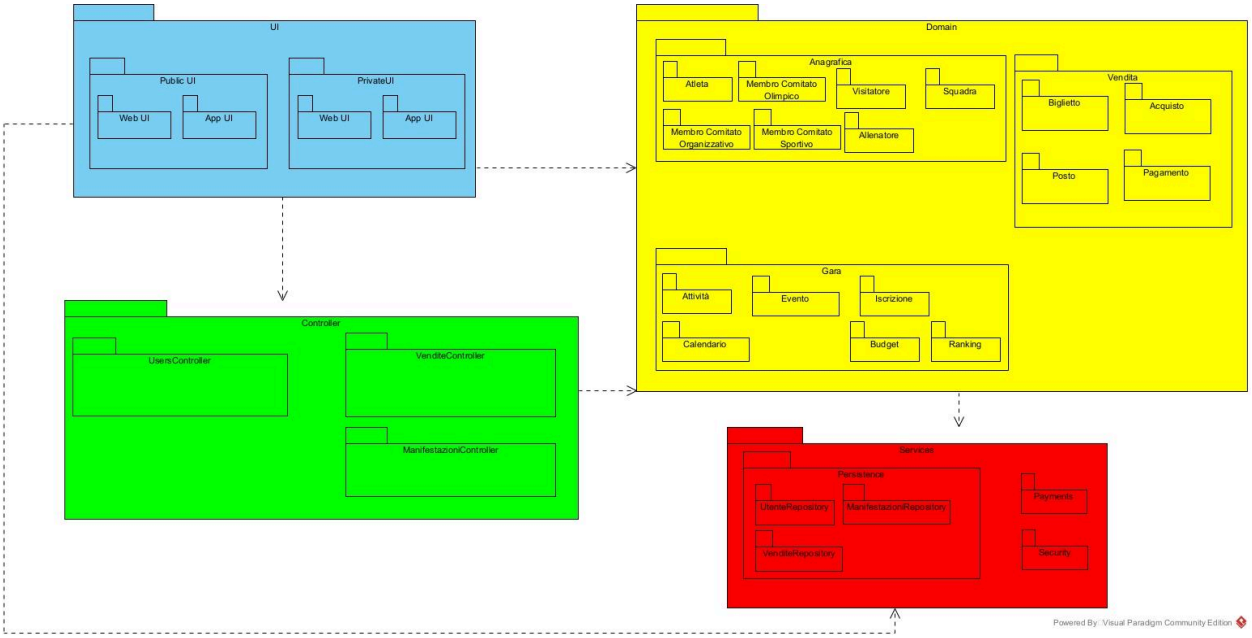


Diagramma delle Attività: AutenticaUtente



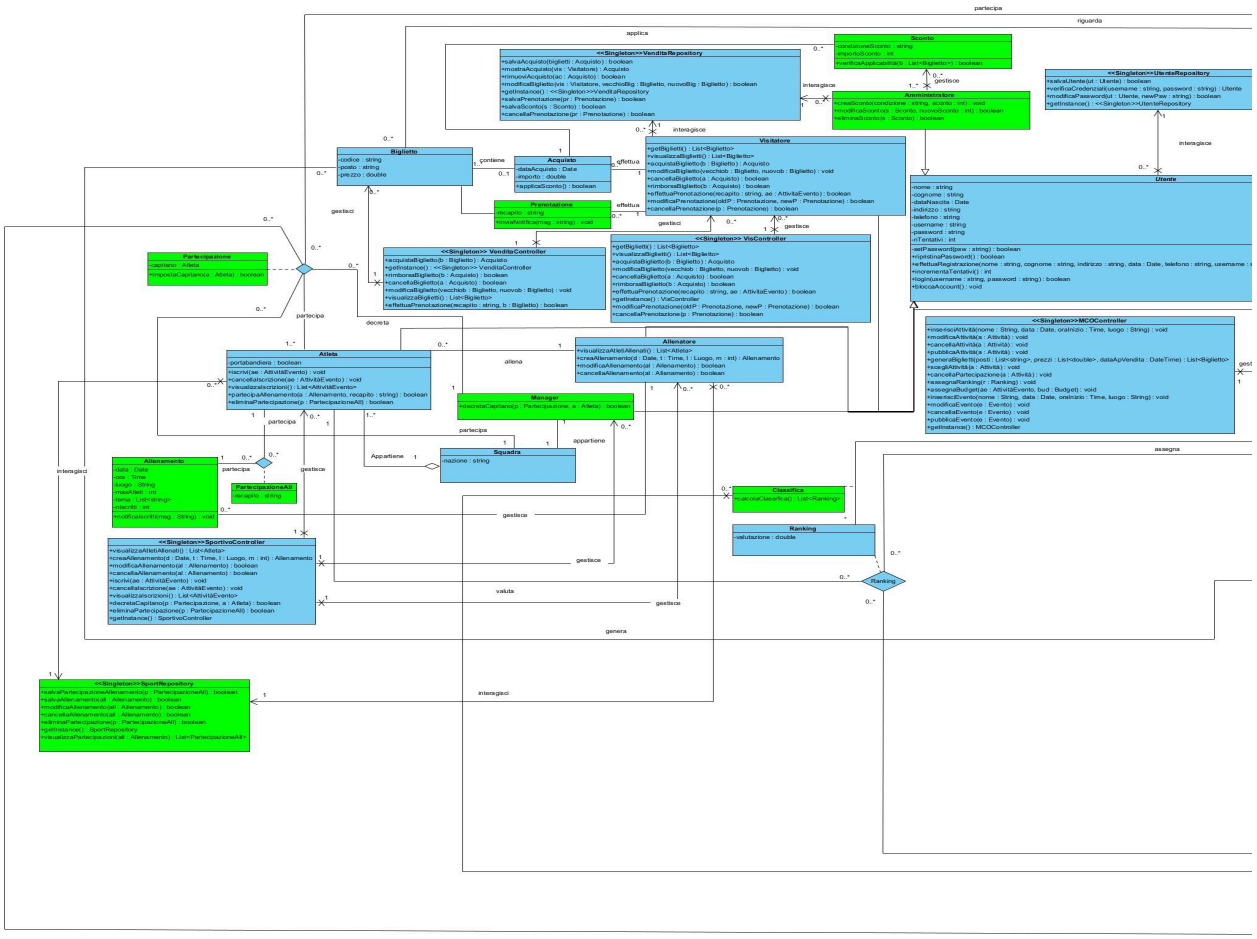
7. Diagramma dei Package

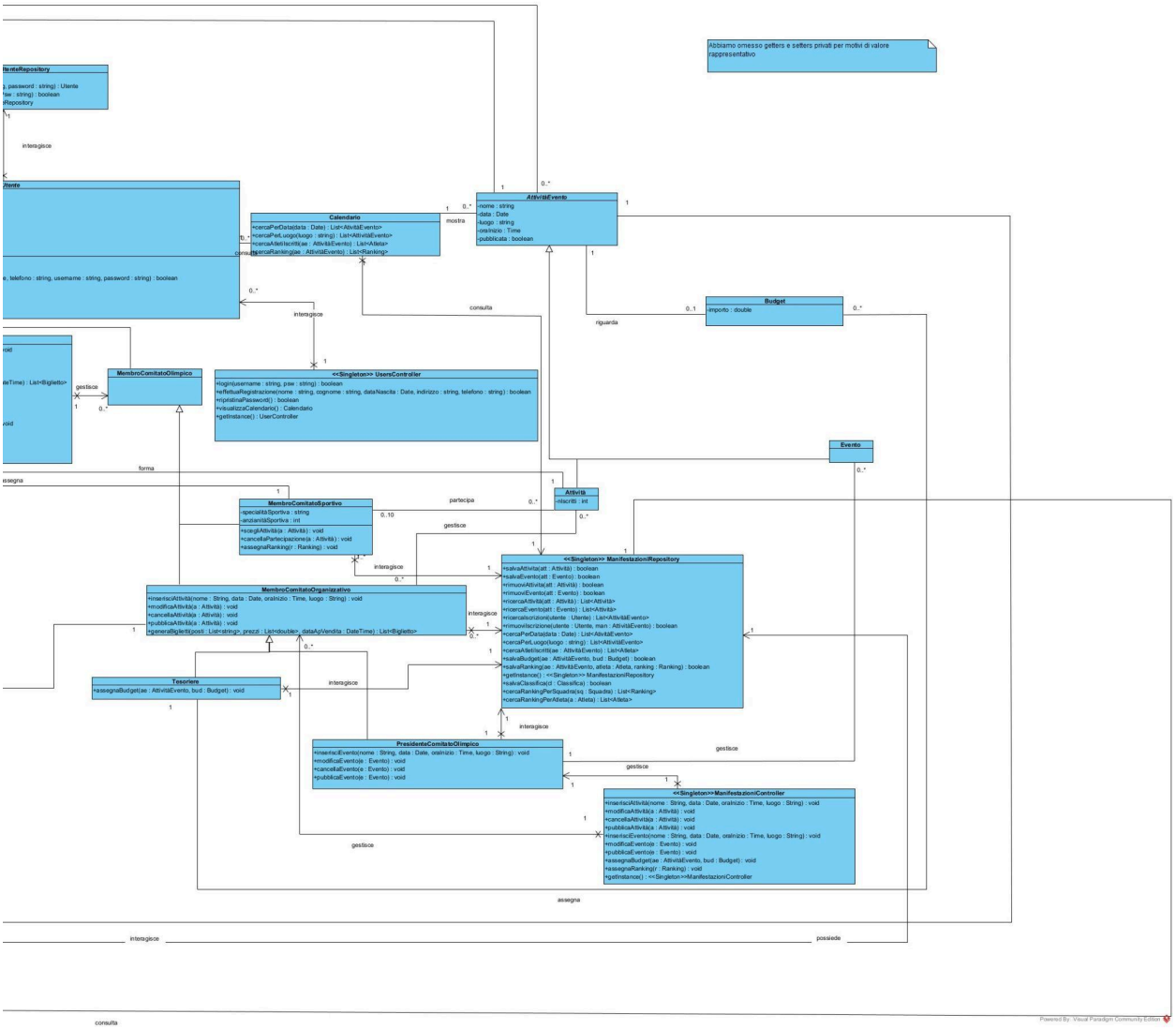
Parte comune:

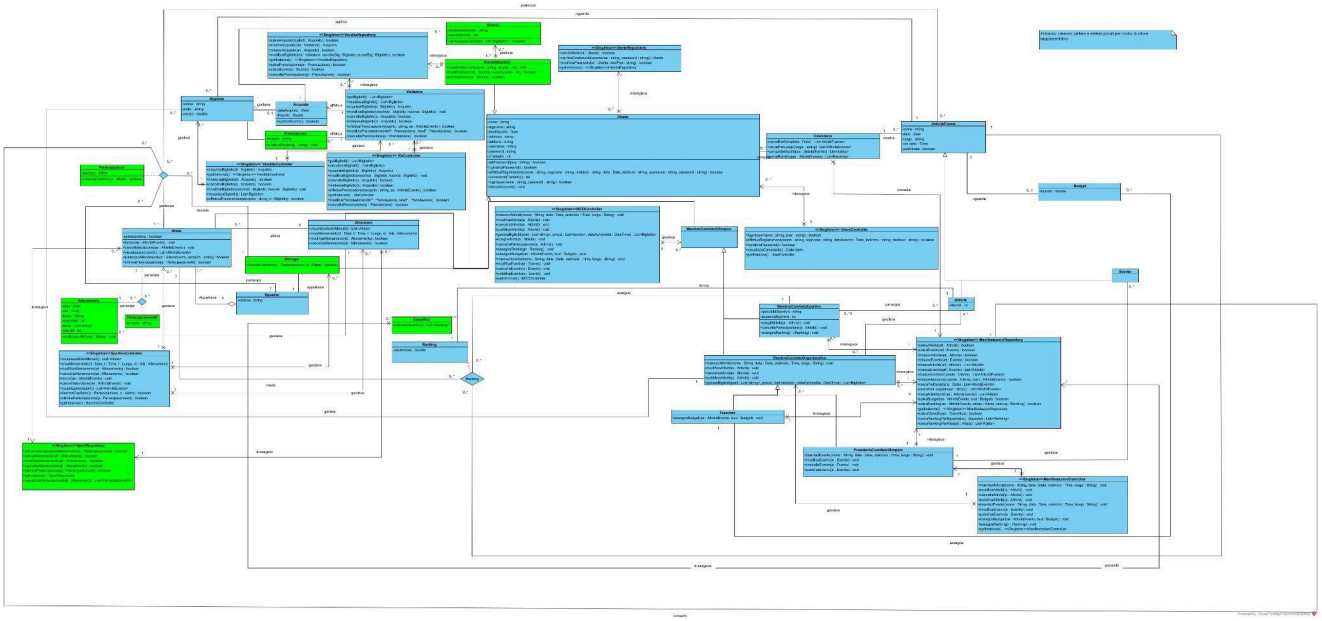


8. Diagramma delle Classi di Progettazione

Autori: Colombi, Garancini


partecipa
riguarda
applica
Sconto
-condizioneSconto : string
-importoSconto : int
+verificaApplicabilità(b : List<Biglietto>) : boolean
<<Singleton>>VenditaRepository
+salvaAcquisto(biglietti : Acquisto) : boolean
+mostraAcquisto(vis : Visitatore) : Acquisto
+rimuoviAcquisto(ac : Acquisto) : boolean
+modificaBiglietto(vis : Visitatore, vecchioBig : Biglietto, nuovoBig : Biglietto) : boolean
+getIstance() : <<Singleton>>VenditaRepository
+salvaPrenotazione(pr : Prenotazione) : boolean
+salvaSconto(s : Sconto) : boolean
+cancellaPrenotazione(pr : Prenotazione) : boolean
interagisce
gestisce
Amministratore
+creaScontoCondizione : string, sconto : int) : void
+modificaSconto(s : Sconto, nuovoSconto : int) : boolean
+eliminaSconto(s : Sconto) : boolean
<<Singleton>>UtenteRepository
+salvaUtente(ut : Utente) : boolean
+verificaCredenziali(username : string, password : string) : Utente
+modificaPassword(ut : Utente, newPsw : string) : boolean
+getIstance() : <<Singleton>>UtenteRepository
interagisce
Biglietto
-codice : string
-posto : string
-prezzo : double
contiene
Acquisto
-dataAcquisto : Date
-importo : double
+applicaSconto() : boolean
effettua
Prenotazione
-acquisto : string
+modificaStato(nuovo : string) : void
effettua
Visitatore
+getBiglietti() : List<Biglietto>
+visualizzaBiglietti() : List<Biglietto>
+acquistaBiglietto(b : Biglietto) : Acquisto
+modificaBiglietto(vecchiob : Biglietto, nuovob : Biglietto) : void
+cancellaBiglietto(a : Acquisto) : boolean
+rimborsaBiglietto(b : Acquisto) : boolean
+effettuaPrenotazione(recapito : string, ae : AttivitàEvento) : boolean
+modificaPrenotazione(oldP : Prenotazione, newP : Prenotazione) : boolean
+cancellaPrenotazione(p : Prenotazione) : boolean
gestisce
Utente
-nome : string
-cognome : string
-dataNascita : Date
-indirizzo : string
-telefono : string
-username : string
-password : string
-nTentativi : int
-setPassword(psw : string) : boolean
+ripristinaPassword() : boolean
+effettuaRegistrazione(nome : string, cognome : string, indirizzo : string, data : Date, telefono : string, username :
+incrementaTentativi() : int
+loginUsername : string, password : string) : boolean
+bloccaAccount() : void
gestisce
Partecipazione
-capitano : Atleta
+imposta Capitano(a : Atleta) : boolean
<<Singleton>> VenditaController
+acquistaBiglietto(b : Biglietto) : Acquisto
+getIstance() : <<Singleton>> VenditaController
+rimborsaBiglietto(b : Acquisto) : boolean
+cancellaBiglietto(a : Acquisto) : boolean
+modificaBiglietto(vecchiob : Biglietto, nuovob : Biglietto) : void
+visualizzaBiglietti() : List<Biglietto>
+effettuaPrenotazione(recapito : string, b : Biglietto) : boolean
<<Singleton>> VisController
+getBiglietti() : List<Biglietto>
+visualizzaBiglietti() : List<Biglietto>
+acquistaBiglietto(b : Biglietto) : Acquisto
+modificaBiglietto(vecchiob : Biglietto, nuovob : Biglietto) : void
+cancellaBiglietto(a : Acquisto) : boolean
+rimborsaBiglietto(b : Acquisto) : boolean
+effettuaPrenotazione(recapito : string, ae : AttivitàEvento) : boolean
+getIstance() : VisController
+modificaPrenotazione(oldP : Prenotazione, newP : Prenotazione) : boolean
+cancellaPrenotazione(p : Prenotazione) : boolean
<<Singleton>>MCOController
+inserisciAttività(nome : String, data : Date, oraInizio : Time, luogo : String) : void
+modificaAttività(a : Attività) : void
+cancellaAttività(a : Attività) : void
+pubblicaAttività(a : Attività) : void
+generaBiglietti(posti : List<string>, prezzi : List<double>, dataApVendita : DateTime) : List<Biglietto>
+sceglAttività(a : Attività) : void
+cancellaPartecipazione(a : Attività) : void
+assegnaRanking(r : Ranking) : void
+assegnaBudget(ae : AttivitàEvento, bud : Budget) : void
+inserisciEvento(nome : String, data : Date, oraInizio : Time, luogo : String) : void
+modificaEvento(e : Evento) : void
+cancellaEvento(e : Evento) : void
+pubblicaEvento(e : Evento) : void
+getIstanza() : MCOController
Atleta
-portabandiera : boolean
+iscrivi(ae : AttivitàEvento) : void
+cancellaIscrizione(ae : AttivitàEvento) : void
+visualizzaIscrizioni() : List<AttivitàEvento>
+partecipaAllenamento(a : Allenamento, recapito : string) : boolean
+eliminaPartecipazione(p : PartecipazioneAtl) : boolean
partecipa
decreta
Allenatore
+visualizzaAtletiAllenati() : List<Atleta>
+creaAllenamento(d : Date, t : Time, l : Luogo, m : int) : Allenamento
+modificaAllenamento(al : Allenamento) : boolean
+cancellaAllenamento(al : Allenamento) : boolean
allena
Manager
+decretaCapitano(p : Partecipazione a : Atleta) : boolean
partecipa
appartiene
Squadra
-nazione : string
Appartiene
Allenamento
-data : Date
-ora : Time
-luogo : String
-maxAtleti : int
-tema : List<string>
-iniziato : int
+modificaIscritti(msg : String) : void
partecipa
ParticipazioneAtl
-recapito : string
gestisce
Classifica
+calcolaClassifica() : List<Ranking>
Ranking
-valutazione : double
Ranking
assegna
<<Singleton>>SportivoController
+visualizzaAtletiAllenati() : List<Atleta>
+creaAllenamento(d : Date, t : Time, l : Luogo, m : int) : Allenamento
+modificaAllenamento(al : Allenamento) : boolean
+cancellaAllenamento(al : Allenamento) : boolean
+iscrivi(ae : AttivitàEvento) : void
+cancellaIscrizione(ae : AttivitàEvento) : void
+visualizzaIscrizioni() : List<AttivitàEvento>
+decretaCapitano(p : Partecipazione, a : Atleta) : boolean
+eliminaPartecipazione(p : PartecipazioneAtl) : boolean
+getIstance() : SportivoController
valuta
gestisce
genera
interagisci
<<Singleton>>SportRepository
+salvaPartecipazioneAllenamento(p : PartecipazioneAtl) : boolean
+salvaAllenamento(al : Allenamento) : boolean
+modificaAllenamento(al : Allenamento) : boolean
+cancellaAllenamento(al : Allenamento) : boolean
+eliminaPartecipazione(p : PartecipazioneAtl) : boolean
+getIstance() : SportRepository
+visualizzaPartecipazioni(al : Allenamento) : List<PartecipazioneAtl>
interagisci

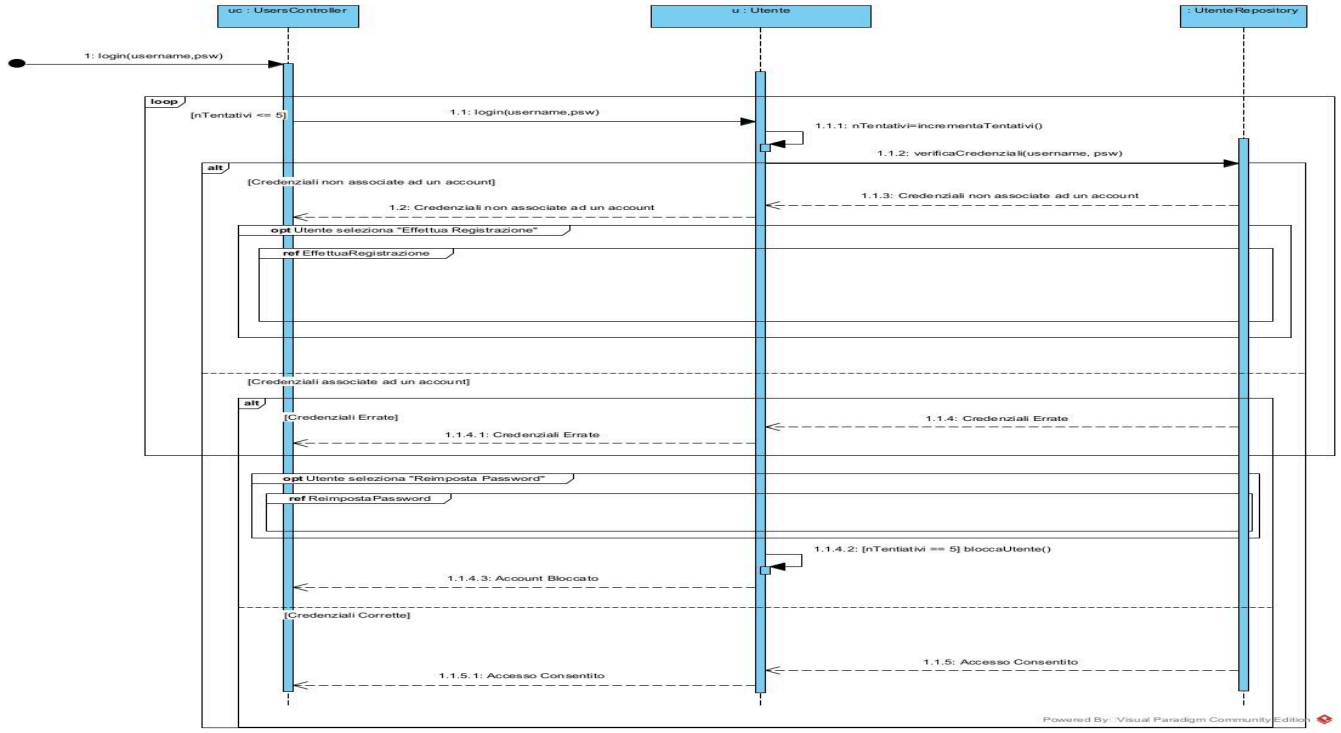

Abbiamo omesso getters e setters privati per motivi di valore rappresentativo
interagisce
mostra
consulta
interagisce
riguarda
gestisce
forma
partecipa
gestisce
interagisce
interagisce
interagisce
interagisce
gestisce
gestisce
gestisce
assegna
interagisce
possiede
consulta
Powered By: Visual Paradigm Community Edition
Calendario
+cercaPerData(data : Date) : List<AttivitàEvento>
+cercaPerLuogo(luogo : string) : List<AttivitàEvento>
+cercaAtletiIscritti(ae : AttivitàEvento) : List<Atleta>
+generaRanking(ae : AttivitàEvento) : List<Ranking>
AttivitàEvento
-nome : string
-data : Date
-luogo : string
-oraInizio : Time
-pubblicata : boolean
Budget
-importo : double
MembroComitatoOlimpico
<<Singleton>> UsersController
+login(username : string, psw : string) : boolean
+effettuaRegistrazione(nome : string, cognome : string, dataNascita : Date, indirizzo : string, telefono : string) : boolean
+ripristinaPassword() : boolean
+visualizzaCalendario() : Calendario
+getIstance() : UserController
Evento
Attività
-risultati : int
MembroComitatoSportivo
-specialitàSportiva : string
-anzianitàSportiva : int
+scegliAttività(a : Attività) : void
+cancellaPartecipazione(a : Attività) : void
+assegnaRanking(r : Ranking) : void
<<Singleton>> ManifestazioniRepository
+salvaAttività(att : Attività) : boolean
+salvaEvento(att : Evento) : boolean
+rimuoviAttività(att : Attività) : boolean
+rimuoviEvento(att : Evento) : boolean
+ricercaAttività(att : Attività) : List<Attività>
+ricercaEvento(att : Evento) : List<Attività>
+ricercaIscrizioniUtente(ut : Utente) : List<AttivitàEvento>
+rimuoviIscrizione(utente : Utente, man : AttivitàEvento) : boolean
+cercaPerData(data : Date) : List<AttivitàEvento>
+cercaPerLuogo(luogo : string) : List<AttivitàEvento>
+cercaAtletiIscritti(ae : AttivitàEvento) : List<Atleta>
+salvaBudget(ae : AttivitàEvento, bud : Budget) : boolean
+salvaRanking(ae : AttivitàEvento, atleta : Atleta, ranking : Ranking) : boolean
+getIstance() : <<Singleton>> ManifestazioniRepository
+salvaClassifica(c : Classifica) : boolean
+cercaRankingPerSquadra(sq : Squadra) : List<Ranking>
+cercaRankingPerAtleta(a : Atleta) : List<Atleta>
MembroComitatoOrganizzativo
+inserisciAttività(nome : String, data : Date, oraInizio : Time, luogo : String) : void
+modificaAttività(a : Attività) : void
+cancellaAttività(a : Attività) : void
+pubblicaAttività(a : Attività) : void
+generaBiglietti(costi : List<string>, prezzi : List<double>, dataApVendita : DateTime) : List<Biglietto>
Tesoriere
+assegnaBudget(ae : AttivitàEvento, bud : Budget) : void
PresidenteComitatoOlimpico
+inseriscEvento(nome : String, data : Date, oraInizio : Time, luogo : String) : void
+modificaEvento(e : Evento) : void
+cancellaEvento(e : Evento) : void
+pubblicaEvento(e : Evento) : void
<<Singleton>>ManifestazioniController
+inseriscAttività(nome : String, data : Date, oraInizio : Time, luogo : String) : void
+modificaAttività(a : Attività) : void
+cancellaAttività(a : Attività) : void
+pubblicaAttività(a : Attività) : void
+inseriscEvento(nome : String, data : Date, oraInizio : Time, luogo : String) : void
+modificaEvento(e : Evento) : void
+pubblicaEvento(e : Evento) : void
+assegnaBudget(ae : AttivitàEvento, bud : Budget) : void
+assegnaRanking(r : Ranking) : void
+getIstance() : <<Singleton>>ManifestazioniController



9. Diagrammi di Sequenza

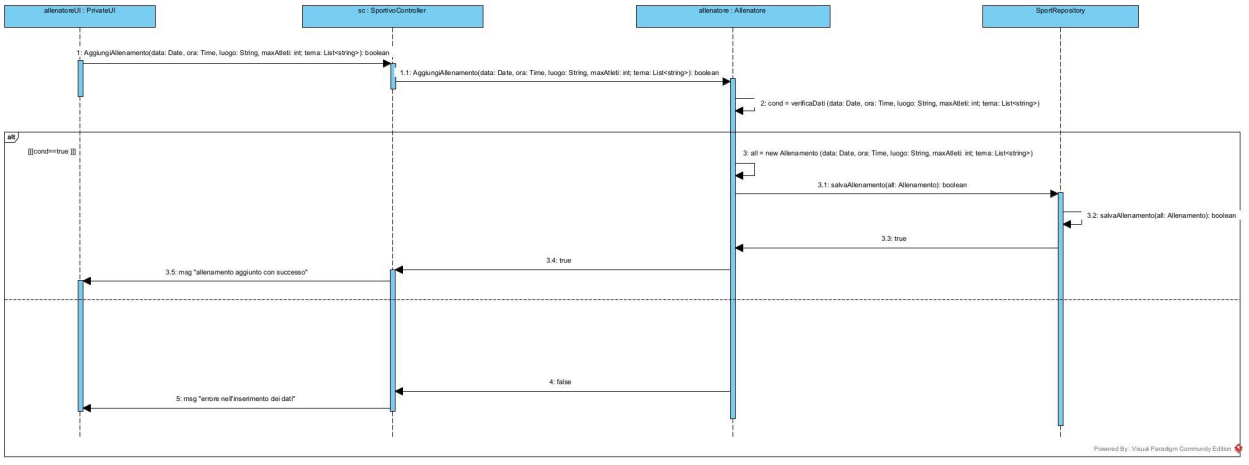
AutenticaUtente

Autore: in comune



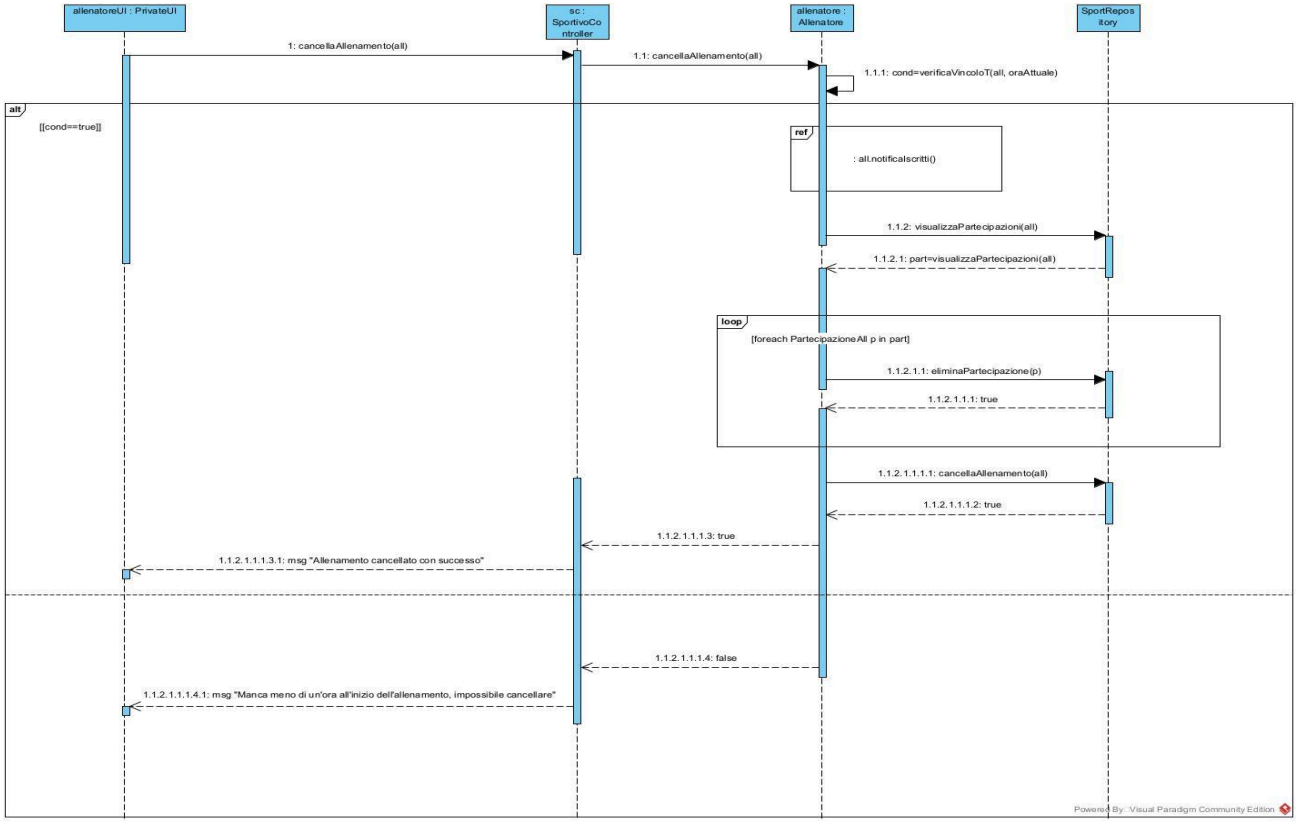
AggiungiAllenamento

Autore: Cestrone



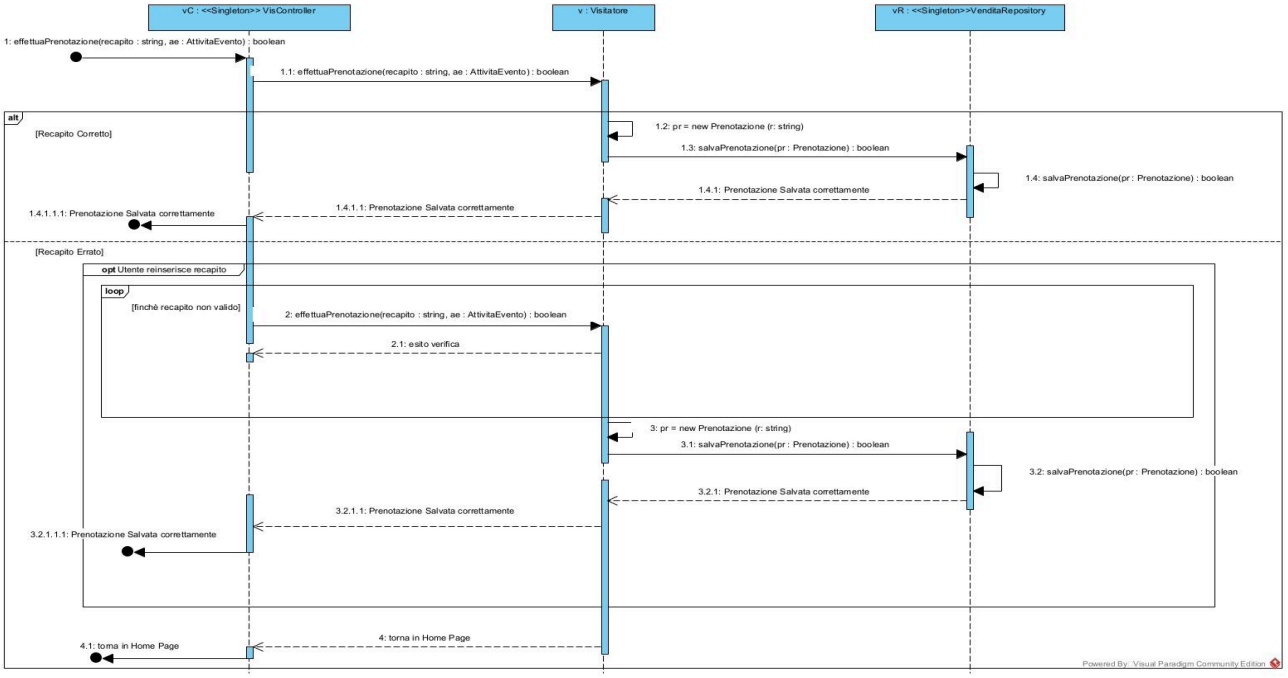
CancellaAllenamento

Autore: Colombi



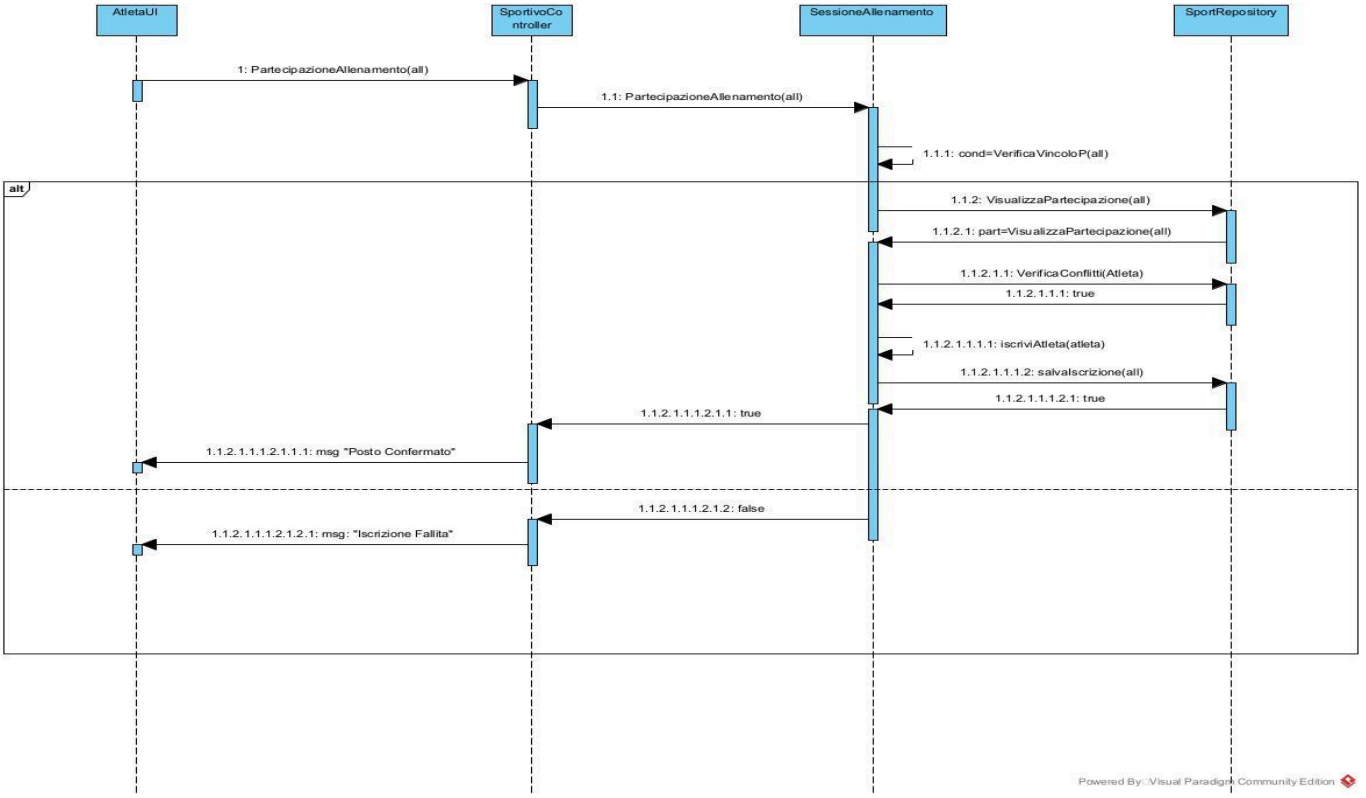
EffettuaPrenotazione

Autore: Garancini



PartecipaAllenamento

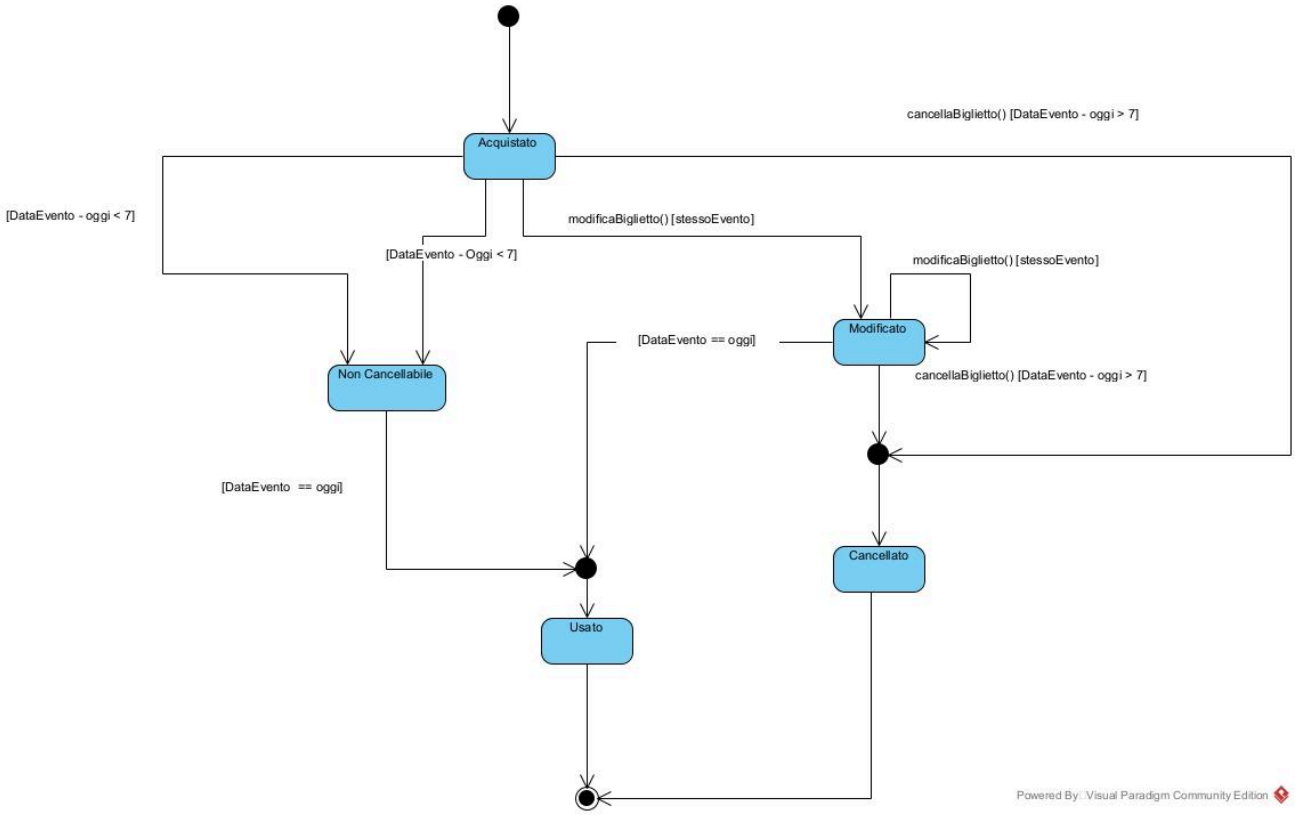
Autore: Gaspani



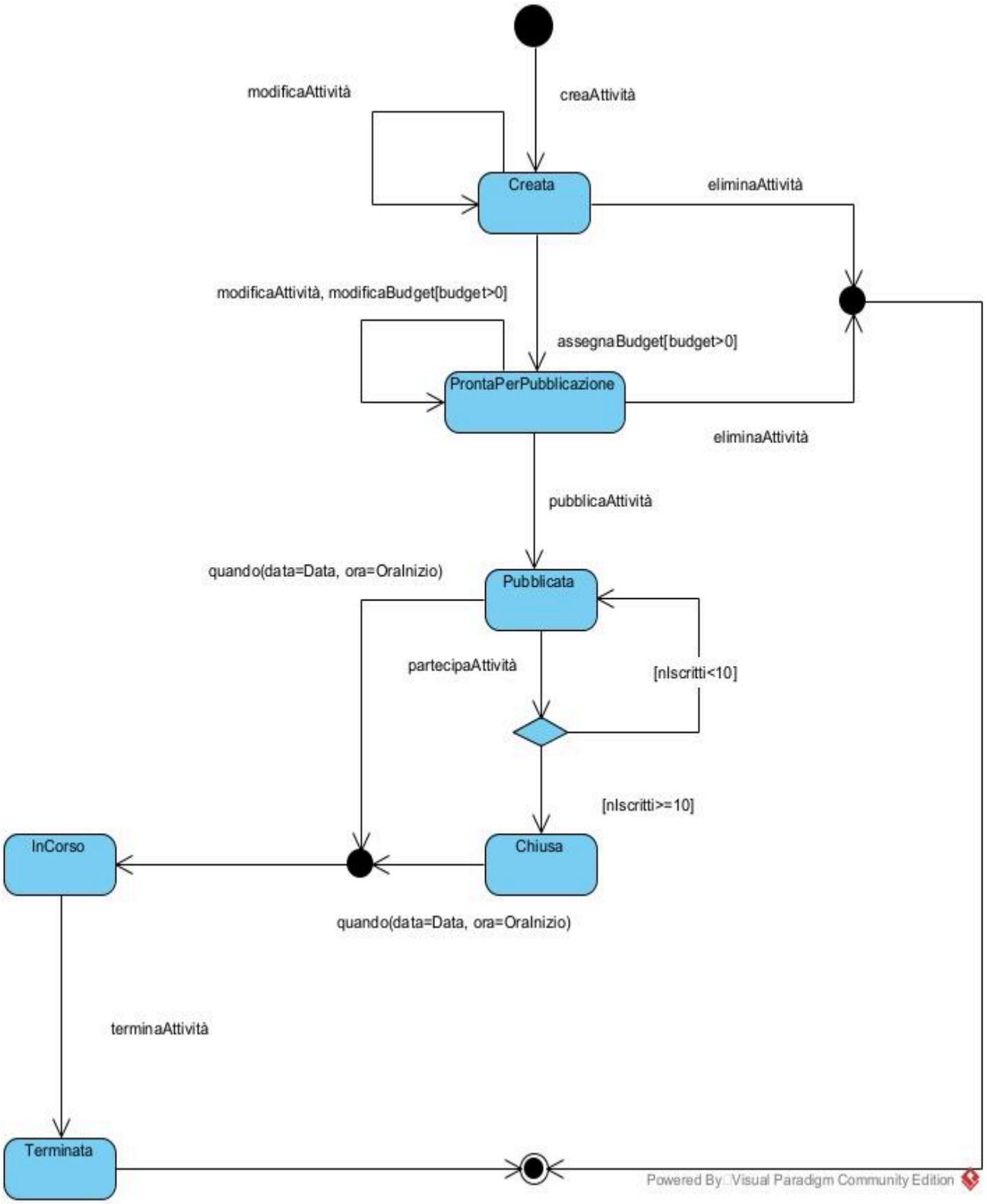
10. Macchine a Stati Finiti

Parte Comune:

Macchina a stati finiti dell'oggetto Biglietto



Macchina a Stati Finiti dell'Oggetto Attività



11. Diagrammi di Comunicazione

Diagramma di Comunicazione: InserisciAttività

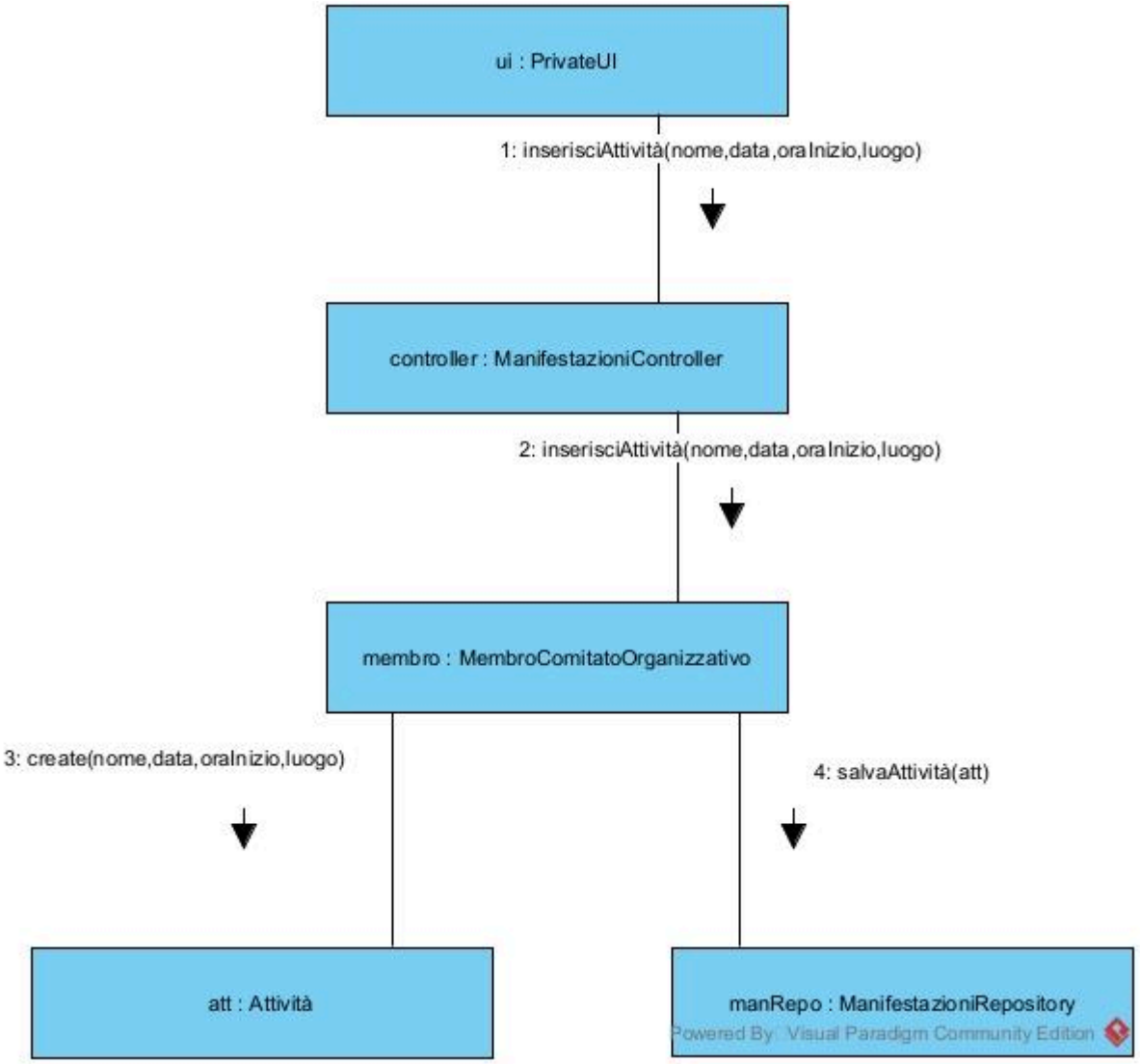
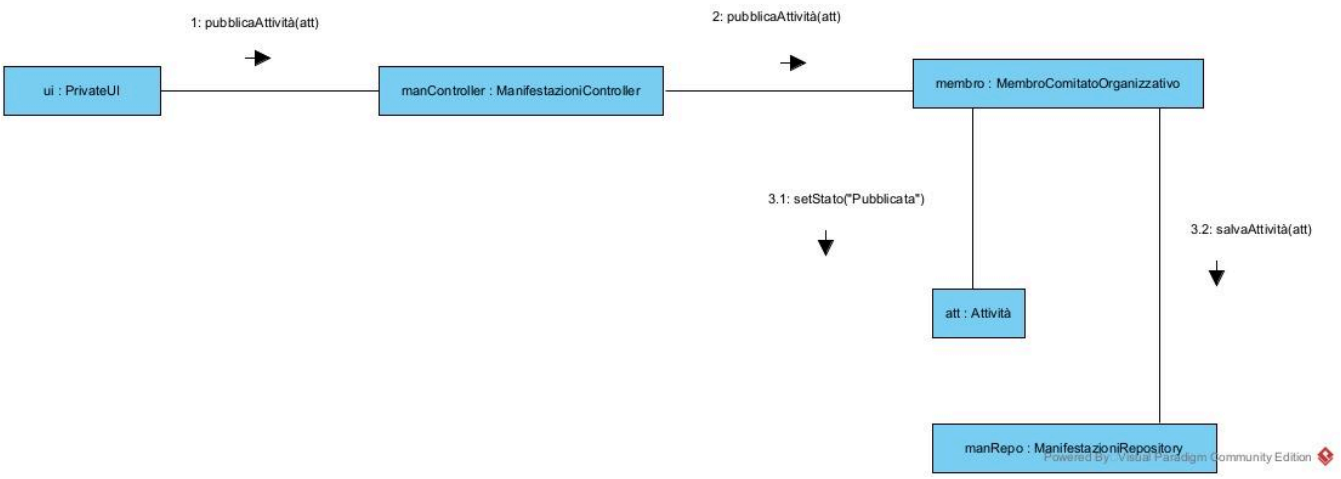


Diagramma di Comunicazione: PubblicaAttività



12.Pattern GRASP e GOF

Design Pattern

Singleton

Descrizione: Le classi Controller e Repository sono classi singleton che implementano un metodo getInstance() per restituire un'istanza utilizzabile per accedere ai metodi relativi alla classe.
Classi del diagramma coinvolte:

- **ManifestazioneController**
- **MembroComitatoController**
- **UserController**
- **VenditaController**
- **ManifestazioneRepository**
- **MembroComitatoRepository**
- **UtenteRepository**
- **VenditaRepository**
- **SportRepository**
- **SportivoController**
- **VisController**
- **MCOController**

Pattern GRASP

Information Expert

Descrizione: Le classi considerate Information Expert hanno al loro interno i dati necessari per svolgere determinate operazioni, rendendole le candidate migliori per questo tipo di responsabilità.
Classi del diagramma coinvolte:

- **ManifestazioneRepository:** tramite la funzione di recupero tramite filtro dei ranking in base all'atleta o alla squadra.
- **PresidenteComitatoOlimpico:** essendo la classe incaricata per la creazione di oggetti di tipo Evento, possiede tutte le informazioni necessarie per effettuare le operazioni di modifica e cancellazione.
- **MembroComitatoOrganizzativo:** essendo la classe incaricata per la creazione di oggetti di tipo Attività, possiede tutte le informazioni necessarie per effettuare le operazioni di modifica e cancellazione.
- **Visitatore:** essendo la classe incaricata per la creazione di oggetti di tipo Prenotazione e Acquisto, possiede tutte le informazioni necessarie per effettuare le operazioni di modifica e cancellazione.

- **Allenatore:** essendo la classe incaricata per la creazione di oggetti di tipo Allenamento, possiede tutte le informazioni necessarie per effettuare le operazioni di modifica e cancellazione.
- **Amministratore:** essendo la classe incaricata per la creazione di oggetti di tipo Sconto, possiede tutte le informazioni necessarie per effettuare le operazioni di modifica e cancellazione.

Creator

Descrizione: Le classi considerate Creator devono avere tutte le informazioni necessarie per istanziare oggetti delle classi di cui creano gli oggetti e devono avere delle forti relazioni tra di esse (aggregazione, associazioni ecc.)
Classi del diagramma coinvolte:

- **Attività:** crea e gestisce oggetti di tipo Ranking
- **Visitatore:** crea e gestisce oggetti di tipo Acquisto e Prenotazione
- **Tesoriere:** crea ed associa oggetti di tipo budget ad oggetti di tipo Attività ed Evento
- **MembroComitatoOrganizzativo:** crea e gestisce oggetti di tipo Attività e di tipo Biglietto
- **PresidenteComitatoOlimpico:** crea e gestisce oggetti di tipo Evento e di tipo Biglietto
- **Squadra:** crea oggetti di tipo Atleta e li aggrega per formare una squadra
- **Allenatore:** crea e gestisce oggetti di tipo Allenamento

Low-Coupling

Descrizione: Riduce dipendenze tra classi, aumentando manutenibilità e riutilizzo. La separazione logica di UI, Controller, Domain e Services permette una riduzione significativa di accoppiamento, problemi di sicurezza e permette deploy e modifica indipendente nelle sottoclassi coinvolte.
Classi del diagramma coinvolte:

- **UI**
- **Controller**
    - **UserController**
    - **VenditaController**
    - **SportivoController**
    - **ManifestazioniController**
    - **MCOController**
- **Domain**
    - **Tutte le classi che fanno parte del modello di dominio**

- **Services**
    - **SportRepository**
    - **VenditaRepository**
    - **ManifestazioniRepository**
    - **UtenteRepository**

High Cohesion

Descrizione: Classi create con finalità specifiche per permettere una maggiore separazione dei compiti, cosa che permette minore accoppiamento tra classi e una maggiore indipendenza a livello di manutenibilità ed eventuale espansione.
Classi del diagramma coinvolte:
Le classi repository sono adibite alla persistenza di sezioni specifiche del progetto:

- **SportRepository**
- **UtenteRepository**
- **ManifestazioniRepository**
- **VenditaRepository**

Le classi controller sono differenziate in base al tipo di utente o al tipo di dati trattati:

- **UsersController**
- **MCOController (Controller per Membro del Comitato Olimpico)**
- **VenditaController**
- **VisController (Controller per visitatore)**
- **ManifestazioniController**
- **SportivoController**

Controller

Descrizione: Le classi Controller sono classi specifiche adibite al controllo e al trasferimento dei dati dallo strato UI allo strato Domain, servono per gestire le transizioni dei dati tra i due livelli

Classi del diagramma coinvolte:

- **UsersController**
- **MCOController**
- **VenditaController**
- **VisController**
- **ManifestazioniController**
- **SportivoController**

Polymorphism

Descrizione: Le classi che ereditano da una superclasse rendono più agevole la manutenzione del codice e permettono di aggiungere delle funzionalità in base al tipo di dato effettivo

Classi del diagramma coinvolte:

- **Utente**
  - **Visitatore**
  - **Allenatore**
  - **Atleta**
  - **MembroComitatoOlimpico**
    - **MembroComitatoSportivo**
    - **MembroComitatoOrganizzativo**
      - **Tesoriere**
      - **PresidenteComitatoOlimpico**
  - **Manager**
- **AttivitàEvento**
  - **Attività**
  - **Evento**